UNIVERSIDAD CONTINENTAL

Escuela Académico Profesional de Ingeniería de Sistemas e Informática

Taller de Proyectos 1 · Semana 13

Aplicación de criterios de calidad de software al proyecto Green AI

Evaluación con el modelo FURPS+ y OWASP Top 10, plan de mejora y validación

INTEGRANTES

Ricardo Porras Veli
Leonid Kisley Orlando Huamán Collazos
Gina Diana Barrera Peña
Jhonny Cristhian Quintana Carhuaricra
Claudio (Claude de Anthropic, Opus 5.5) – agente de IA que ejecutó las auditorías y la ola 1 bajo supervisión del equipo

DOCENTE

Jorge Alfredo Guevara Jiménez

NRC

30170

Huancayo, noviembre de 2026

1. Introducción y logro de la sesión

El desarrollo de soluciones de software contemporáneas para infraestructuras críticas demanda trascender la mera verificación de funcionalidad algorítmica. En proyectos orientados a la sostenibilidad computacional y eficiencia energética en centros de datos, como Green AI, la confiabilidad del cálculo, la protección perimetral frente a intrusiones externas, la operabilidad ágil y la mantenibilidad a largo plazo constituyen atributos de primer orden que determinan la viabilidad operativa del producto.

El logro de aprendizaje establecido para la Semana 13 estipula: «Al finalizar la sesión, el estudiante aplica criterios de calidad de software con claridad y propósito al proyecto». En concordancia con este objetivo, el equipo de ingeniería del proyecto Green AI ejecutó un proceso integral de evaluación, mejora sistemática y validación formal de calidad y seguridad, sustentado en dos de los marcos metodológicos más consolidados de la industria:

1. El modelo de factores de calidad FURPS+ (Hewlett-Packard / Rational Software Corp.), adaptado a los lineamientos institucionales de la Universidad Continental.
2. El estándar de seguridad de aplicaciones web y APIs OWASP Top 10 (edición 2025, https://top10.owasp.org/2025/).

El presente documento sistematiza los resultados obtenidos en las tres fases del proceso metodológico: el diagnóstico inicial de calidad y seguridad, la formulación de los planes de mejora por olas de trabajo, la implementación verificable de la primera ola de mitigación (Ola 1), la auditoría de implementación independiente y el dictamen final integral. Asimismo, se incorpora la matriz de calidad docente con 50 preguntas de control técnico y la justificación explícita de las brechas que separan el estado actual del sistema respecto a la meta esperada.

2. Por qué aplicar calidad de software

La aplicación sistemática de criterios de calidad de software adquiere un papel neurálgico en Green AI debido a la naturaleza de su dominio. El sistema tiene por finalidad optimizar la capacidad y el consumo eléctrico de centros de datos mediante el monitoreo de telemetría de 50 servidores agrupados en racks y la inferencia anticipada de demanda de CPU a 15 minutos mediante modelos de Machine Learning (Random Forest).

Si un sistema de esta índole careciera de rigurosos controles de calidad, se enfrentarían riesgos severos:

1. Impacto operacional por falsas recomendaciones: Un error de cálculo en las políticas de recomendación o una falla en las restricciones operativas (latencia, memoria, tasa de errores) podría provocar la desactivación errónea de servidores en momentos de alta concurrencia, ocasionando degradación de servicio y penalizaciones por acuerdos de nivel de servicio (SLA).
2. Vulnerabilidad perimetral en infraestructuras híbridas: Al operar mediante microservicios distribuidos (Java Spring Boot, Python FastAPI, Go) y comunicarse con servicios en la nube (Supabase para autenticación y persistencia histórica, Render para despliegue de contenedores, GitHub Pages para distribución estática), una brecha en la validación de tokens JWT o configuraciones permisivas en el control de acceso a la base de datos expondría registros operacionales y credenciales de operadores.
3. Degradación por sobrecarga y tiempos de respuesta inadmisibles: Un diseño ineficiente en los adaptadores de persistencia o la carencia de límites de tasa (rate limiting) ante peticiones masivas de inferencia sobrecargaría los procesadores del backend, elevando paradójicamente el consumo energético que la solución busca mitigar.
4. Falta de transparencia y honestidad experimental: La adopción de afirmaciones no demostradas respecto a la procedencia de la telemetría induciría a error a los evaluadores y operadores. En Green AI, la aplicación de calidad garantiza declarar explícitamente que la telemetría actual proviene de simulación determinista controlada y que la potencia eléctrica corresponde a un modelo matemático lineal, preservando el rigor ético y científico del proyecto.

La calidad de software, por tanto, no representa una etapa cosmética o posterior al desarrollo, sino un principio rector transversal que garantiza que cada componente del monorepo sea determinista, seguro, mantenible y verificable a través de pruebas automatizadas continuas.

3. Modelo FURPS+ y OWASP Top 10

3.1. El Modelo de Calidad FURPS+

Propuesto originalmente por Robert Grady en Hewlett-Packard y adoptado formalmente por Rational Software, el modelo FURPS+ clasifica los atributos y requisitos de calidad de software en cinco categorías fundamentales, complementadas por factores de restricción («+»):

- F - Funcionalidad (Functionality): Evalúa la completitud del conjunto de características del sistema, las capacidades operacionales de los casos de uso, la seguridad de las transacciones y la consistencia del procesamiento de negocio. En Green AI, abarca los casos de uso del Producto Mínimo Viable (PMV1 y PMV2), tales como visualización de telemetría en tiempo real (HU-01), histórico paginado (HU-02), predicción de CPU a 15 minutos (HU-03), comparación de modelos (HU-04), experimentos en MLflow (HU-05), recomendaciones con restricciones (HU-06) y control de acceso RBAC (HU-14).
- U - Usabilidad (Usability): Examina los factores humanos en la interacción persona-computador, la ergonomía visual, la coherencia de navegación, la claridad de los mensajes de error, el tiempo requerido para completar tareas críticas y la documentación de ayuda al usuario.
- R - Confiabilidad (Reliability): Valora la capacidad del software para mantener su nivel de servicio en condiciones operativas normales y anómalas. Incluye la tolerancia a fallos, la gestión defensiva de errores mediante formatos estandarizados (RFC 7807), el aislamiento entre servicios mediante timeouts y la cobertura exhaustiva de pruebas unitarias, de integración y de humo.
- P - Rendimiento (Performance): Mide la velocidad de respuesta, el consumo eficiente de memoria RAM y procesador, la huella en disco de las imágenes de contenedor y el comportamiento del sistema ante condiciones de arranque en frío en infraestructuras de nube.
- S - Soporte y Mantenibilidad (Supportability): Analiza la modularidad de la base de código, la observancia de patrones arquitectónicos desacoplados (arquitectura hexagonal), la estabilidad de los contratos de interfaz (OpenAPI), la mantenibilidad del repositorio y la integración continua (CI/CD).
- «+» - Restricciones Adicionales: Incorpora restricciones de diseño (patrones arquitectónicos obligatorios), de implementación (lenguajes, bibliotecas compatibles y estándares de codificación), de interfaz (interacción segura con servicios externos como Supabase), físicas y de despliegue (orquestación mediante Docker Compose, viabilidad de hosting a costo cero en Render y GitHub Pages) y normativas legales (licencias Open Source y Ley N° 29733 de Protección de Datos Personales en el Perú).

3.2. El Estándar OWASP Top 10 (2025)

La iniciativa Open Web Application Security Project (OWASP) publica periódicamente el consenso global de las vulnerabilidades más críticas que amenazan a las aplicaciones web y APIs modernas. Su marco de referencia Top 10:2025 (https://top10.owasp.org/2025/) estructura la seguridad en diez categorías:

1. A01:2025 - Pérdida de Control de Acceso (Broken Access Control): Elusión de privilegios, acceso horizontal/vertical no autorizado, políticas de seguridad a nivel de fila (Row Level Security - RLS) insuficientes y Falsificación de Peticiones del Lado del Servidor (SSRF), capacidad de inducir al servidor a emitir peticiones no autorizadas hacia recursos internos o redes privadas aisladas.
2. A02:2025 - Configuración de Seguridad Incorrecta (Security Misconfiguration): Cabeceras HTTP de protección web ausentes, valores predeterminados inseguros y directivas de control de orígenes (CORS) excesivamente permisivas.
3. A03:2025 - Fallos en la Cadena de Suministro de Software (Software Supply Chain Failures): Inclusión de paquetes desactualizados o comprometidos y carga de scripts externos desde redes de distribución de contenido (CDN) sin comprobación criptográfica de integridad.
4. A04:2025 - Fallos Criptográficos (Cryptographic Failures): Transmisión o almacenamiento de información sensible en texto claro, protocolos obsoletos y manejo inseguro de claves de cifrado.
5. A05:2025 - Inyección (Injection): Fallos derivados de la interpolación de datos no confiables en intérpretes de comandos, consultas SQL/ORM o lenguajes de consulta de monitoreo (PromQL).
6. A06:2025 - Diseño Inseguro (Insecure Design): Ausencia de patrones de defensa en profundidad, modelos de amenazas no atendidos y carencia de límites de consumo de recursos (rate limiting).
7. A07:2025 - Fallos de Autenticación (Authentication Failures): Debilidades en la verificación de tokens de sesión JWT, fijación de sesiones y almacenamiento indebido de credenciales en el cliente.
8. A08:2025 - Fallos de Integridad del Software o de los Datos (Software or Data Integrity Failures): Uso de dependencias y flujos de automatización CI/CD dependientes de etiquetas mutables sin anclaje criptográfico inmutable.
9. A09:2025 - Fallos de Registro y Alertas de Seguridad (Security Logging and Alerting Failures): Omisión de trazas de auditoría ante accesos no autorizados (errores HTTP 401/403) y ausencia de alertas automáticas ante incidentes.
10. A10:2025 - Manejo Inadecuado de Condiciones Excepcionales (Mishandling of Exceptional Conditions): Errores no controlados que filtran detalles internos o dejan al sistema en estado inseguro (fail-open) ante fallos de dependencias, tiempos de espera o entradas anómalas.

Respecto de la edición 2021, los cambios principales son: SSRF (antes A10:2021) fue absorbida por A01; A06:2021 (Componentes Vulnerables y Obsoletos) se amplió a A03:2025 (Cadena de Suministro de Software); se incorporó la categoría nueva A10:2025; y se renombraron A07 (antes «Identification and Authentication Failures») y A09 (antes «Logging and Monitoring»). La auditoría OWASP del proyecto, originalmente elaborada con la lista 2021, fue reclasificada a la edición 2025 según se documenta en docs/calidad/08_reclasificacion_owasp_2025.md.

En el marco metodológico del curso, las diez categorías de OWASP se integran directamente como el factor técnico de seguridad dentro del componente «+» del modelo FURPS+, estableciendo un examen exhaustivo de la robustez del sistema.

4. Metodología de trabajo y distribución de roles

4.1. Flujo Metodológico de 9 Pasos y 3 Fases

La ejecución del trabajo se apegó estrictamente a la «Guía para la Evaluación, Mejora y Validación de la Calidad y Seguridad de Software mediante FURPS+ y OWASP Top 10 utilizando Agentes de IA». Dicha guía estructura el ciclo de vida de aseguramiento de calidad en nueve pasos correlativos divididos en tres fases fundamentales:

- Fase 1: Evaluación y Mejora de Calidad (FURPS+)
  - Paso 1: Auditoría inicial FURPS+ (entregable 01_auditoria_furps.md).
  - Paso 2: Planificación de mejoras de calidad (entregable 02_plan_furps.md).
  - Paso 3: Aplicación controlada del plan por parte del equipo técnico.
  - Paso 4: Auditoría independiente de implementación FURPS+ (entregable 03_auditoria_implementacion_furps.md).
- Fase 2: Evaluación y Mejora de Seguridad (OWASP Top 10)
  - Paso 5: Auditoría inicial de seguridad OWASP (entregable 04_auditoria_owasp.md).
  - Paso 6: Planificación de remediación de vulnerabilidades (entregable 05_plan_owasp.md).
  - Paso 7: Aplicación controlada de las remediaciones programadas.
  - Paso 8: Auditoría independiente de implementación OWASP (entregable 06_auditoria_implementacion_owasp.md).
- Fase 3: Validación Global Integral
  - Paso 9: Auditoría final integral y emisión de dictamen técnico (entregable 07_auditoria_final.md).

4.2. Colaboración Humano-Agente y Corrección de Sobrecalificaciones

Una singularidad del proceso radica en el esquema de trabajo cooperativo entre los integrantes del equipo y los agentes de inteligencia artificial. Claude Code (Anthropic, Opus 5.5) actuó como coordinador metodológico, evaluador crítico e implementador directo de las remediaciones en la máquina de desarrollo local, mientras que Antigravity CLI (con modelo Gemini) fue asignado a la ejecución de borradores analíticos de auditoría y planes detallados.

Para garantizar la honestidad experimental, cada informe generado por un agente fue sometido a revisión cruzada contra el código fuente real del monorepo por parte del equipo y de Claude Code. Este filtro crítico resultó decisivo: la primera auditoría automatizada producida por un agente otorgó a Green AI una calificación del 96.00 % en FURPS+, asumiendo erróneamente que la presencia de maquetas en la interfaz equivalía a funcionalidad completa. Tras contrastar el reporte contra los endpoints y la base de datos, se identificaron brechas esenciales:

- La aprobación humana de recomendaciones operativas (HU-07) carecía de persistencia en base de datos.
- Los reportes históricos solo se exportaban en formato CSV plano, sin generación de documentos PDF formales (HU-13).

- El consumo energético y la potencia se basaban en un modelo matemático sintético, no en lecturas de sensores de hardware físicos en tiempo real (HU-09).
- La interfaz de MLflow operaba de forma local sin despliegue público en la nube (HU-05).

Tras subsanar estas sobrecalificaciones y ceñir la evaluación a la evidencia técnica verificable, la calificación inicial formal de FURPS+ se reajustó de manera transparente a 84.0 % (168 / 200 puntos brutos; 84.78 / 100 ponderado), garantizando un punto de partida verosímil y defendible. En la revisión del equipo del 12/11/2026 se aplicó un último reajuste: M-R01 (errores RFC 7807) pasó de Cumple (5) a Parcial (3) porque el servicio Prediction usa su propio sobre {code, message} en lugar de problem+json, (la primera reducción había dejado 85.0 %, 170 / 200).

4.3. Distribución de Responsabilidades por Integrante

Conforme a los requerimientos de la actividad colaborativa de la Semana 13, los factores del modelo de calidad FURPS+ y el estándar de seguridad OWASP fueron distribuidos formalmente entre los integrantes del equipo de proyecto, asumiendo cada uno la responsabilidad técnica sobre los hallazgos y las acciones correctivas. Los roles corresponden a los definidos en el acta de constitución y la gestión del proyecto (Product Owner, Scrum Master / ML Lead, backend e integración, infraestructura y DevOps):

Factor	Dimensión de Calidad	Integrante Responsable	Rol en el Proyecto	Alcance y Responsabilidad Técnica
F	Funcionalidad	Jhonny Cristhian Quintana Carhuaricra	Product Owner (PO) y líder del proyecto: documentación, pruebas y gobernanza	Verificación de historias de usuario (HU-01 a HU-15), casos de uso de PMV1 y PMV2, alcance de negocio y reporte de experimentación.
U	Usabilidad	Leonid Kisley Orlando Huamán Collazos	Development Team: infraestructura, Docker y observabilidad (Infrastructure & DevOps Engineer), con interfaz UI/UX	Consistencia de interfaz, accesibilidad (ARIA / WCAG), diseño responsivo, ergonomía del mapa de calor de Racks y navegación en <= 3 pasos; además, desde infraestructura, tiempos de respuesta en Render (M-P01) y alojamiento a costo cero (M-PL03).
R	Confiabilidad	Ricardo Porras Veli	Development Team: backend, contratos e integración (Lead Backend & MLOps Engineer)	Estandarización de errores (RFC 7807), sondas de salud (health checks), timeouts escalonados, suites de pruebas automatizadas y contratos de persistencia.
P	Rendimiento	Gina Diana Barrera Peña	Scrum Master y ML Lead (datos y modelos)	Latencias de respuesta, optimización de consumo de memoria RAM, aligeramiento de imágenes Docker de inferencia y profiling de algoritmos.
S	Soporte / Mantenibilidad	Gina Diana Barrera Peña y Jhonny Quintana Carhuaricra	Scrum Master / ML Lead y Product Owner (documentación y pruebas)	Desacoplamiento de arquitectura hexagonal, especificaciones OpenAPI versionadas, pipelines de CI en GitHub Actions e higiene del monorepo.
+	Restricciones Adicionales y OWASP	Ricardo Porras Veli (con apoyo de Claude Code)	Development Team: backend, contratos e integración (Lead Backend	Evaluación de las 10 categorías OWASP Top 10:2025, cabeceras HTTP, Subresource Integrity y anclaje inmutable

Factor	Dimensión de Calidad	Integrante Responsable	Rol en el Proyecto	Alcance y Responsabilidad Técnica
			& MLOps Engineer); coordina la evaluación de seguridad de la sesión	de CI/CD. Los hallazgos OWASP se reparten por dominio: datos y credenciales del dataset con Gina Barrera (OW-A04), sesión en el frontend con Leonid Huamán (OW-A07) y políticas de Supabase con el dueño de esa cuenta (OW-A01, OW-A04).

5. Fase 1 — Calidad de Software (Modelo FURPS+)

5.1. Auditoría Inicial FURPS+ (Entregable 01)

La auditoría formal de calidad de software se documentó en el informe docs/calidad/01_auditoria_furps.md, evaluando los microservicios del monorepo (software/gateway, software/monitoring, software/data-processing, software/prediction, software/simulator, software/frontend y software/infraestructura).

La calificación global inicial ponderada alcanzó 84.78 / 100 puntos (168 de 200 puntos brutos, 84.0 %, en la matriz de 40 preguntas), con el siguiente desglose por categoría. Nota de transparencia: el valor «85.00 / 100 ponderado» publicado antes no seguía la ponderación (era el bruto 170 / 200; con el R anterior habría sido 86.12), mientras que el 87.62 posterior sí era correcto con ese R; ambos se recalcularon tras reajustar M-R01 (84.78 inicial y 86.28 post Ola 1).

Factor FURPS+	Ponderación	Puntaje Obtenido (0-100)	Nivel Cualitativo	Principales Brechas Identificadas
Funcionalidad (F)	25 %	78 / 100	Aceptable	Aprobación humana de recomendaciones sin persistencia (HU-07), potencia eléctrica modelada sin sensores físicos (HU-09), MLflow UI sin exposición en nube (HU-05) y reporte histórico sin soporte PDF formal (HU-13).
Usabilidad (U)	15 %	80 / 100	Aceptable	Ausencia de una guía rápida de usuario para la operación del sistema (M-U07), falta de escaneo formal WCAG 2.1 AA con Lighthouse y flujo de cálculo de energía en más de 3 pasos (M-U08).
Confiabilidad (R)	20 %	87 / 100	Bueno	Cobertura con 415+ pruebas y problem+json en Gateway, Data Processing y Monitoring; Prediction usa aún su propio sobre {code, message} (M-R01 Parcial, migración prevista el 23/11/2026). Brecha adicional: riesgo de políticas permisivas en la base de datos externa de Supabase ante saturación.
Rendimiento (P)	15 %	87 / 100	Bueno	Latencias locales inferiores a 30 ms en Java y 4 ms en Go. Brecha: retardo por arranque en frío (cold start de ~60 s) en Render Free Tier tras 15 minutos de inactividad (M-P01).
Soporte (S)	15 %	90 / 100	Bueno	Monorepo limpio sin gitlinks ni repositorios anidados, CI automatizado en GitHub Actions y contratos OpenAPI vigentes. Brecha: acoplamiento de adaptadores en runner.py del Simulator.
Restricciones (+)	10 %	93 / 100	Bueno	Punto único perimetral, orquestación unificada con Docker Compose, compatibilidad de licencias Open Source y apego estricto a la Ley N° 29733. Brecha:

Factor FURPS+	Ponderación	Puntaje Obtenido (0-100)	Nivel Cualitativo	Principales Brechas Identificadas
				cuota compartida de 750 h/mes en Render.
TOTAL PONDERADO	**100 %**	**84.78 / 100**	**Satisfactorio**	**El sistema cumple las condiciones funcionales requeridas, requiriendo un plan de mejora por fases.**

5.2. Plan de Mejora FURPS+ (Entregable 02)

El plan integral de mejora de calidad se estructuró en docs/calidad/02_plan_furps.md, fundamentándose en dos principios de diseño no negociables: 1. El sistema ya funciona: Se prohíbe cualquier refactorización agresiva o cambio de contratos que altere el comportamiento de los microservicios en producción. 2. Barrera de regresión obligatoria: Toda tarea exige la ejecución de su suite de pruebas correspondiente antes de su integración, garantizando cero regresiones funcionales sobre la línea base de más de 415 pruebas automatizadas: * Frontend: 48 pruebas en node:test. * Prediction Service: 100 pruebas en pytest. * Data Processing Service: 121 pruebas en pytest. * Monitoring Service: 112 pruebas en JUnit 5 / Maven. * API Gateway: 12 pruebas en Spring Cloud Gateway y 30 pruebas en Go test. * Simulator: 10 pruebas unitarias deterministas. * Smoke Test Global: 14/14 comprobaciones HTTP sobre el stack en ejecución.

El roadmap de mejora se programó en tres olas de trabajo:

- Ola 1 - Semana 13 (07/11/2026): Implementación inmediata documental y de usabilidad: redacción y publicación de la Guía Rápida de Usuario (M-U07).
- Ola 2 - Sprint 7 (17/11/2026 - 23/11/2026): Funcionalidades críticas y mantenibilidad: persistencia de aprobación de recomendaciones (HU-07 / M-F07), generación de reporte PDF institucional (HU-13 / M-F11), exposición y evidencia de MLflow UI (HU-05 / M-F05), verificación en Staging con sesión real (/dev/), refactorización hexagonal en Simulator (runner.py / M-S01), script keep-alive para precalentamiento en Render (M-P01) y tabla accesible para gráficos de Chart.js (M-U06).
- Ola 3 - Sprint 8 (24/11/2026 - 30/11/2026): Optimización avanzada y cierre de brechas operativas: optimización del flujo de cálculo de energía a tres pasos (M-U08) y desacoplamiento de blueprints de despliegue en Render (M-PL03).

5.3. Aplicación del Plan — Ola 1 (Semana 13)

Durante la Ola 1, el equipo implementó con éxito la mejora M-U07 mediante la creación del documento entregables/guia-de-uso/GUIA-RAPIDA-USUARIO.md.

La guía se redactó bajo los siguientes estándares de calidad: 1. Orientación por roles de usuario: Define explícitamente los alcances operativos para Administradores de TI (ADMIN), Operadores de Infraestructura (OPERATOR) y Observadores/Auditores (VIEWER). 2. Regla de diseño operacional de tres pasos: Cada uno de los 8 flujos de negocio esenciales del Observatorio fue estructurado de manera que el operador pueda completarlo en 3 pasos o menos, empleando las etiquetas y textos exactos de la interfaz gráfica: * Tarea 1: Iniciar sesión o registrarse (register.html / login.html). * Tarea 2: Visualizar métricas agregadas de los 50 servidores (dashboard.html). * Tarea 3: Inspeccionar Racks mediante el mapa de calor interactivo (network.html). * Tarea 4: Predecir la demanda de CPU

a 15 minutos (método directo desde Racks o Centro de Operación). * Tarea 5: Evaluar recomendaciones operativas con restricciones de política. * Tarea 6: Descargar reportes históricos consolidados en formato CSV. * Tarea 7: Conmutar la apariencia visual entre Modo Claro y Modo Oscuro. * Tarea 8: Consultar el significado de indicadores y fórmulas mediante la ayuda contextual (botones «?»). 3. Transparencia y honestidad experimental: La guía incluye una sección final de advertencia ética que aclara al operador que la telemetría es de origen sintético y que el cálculo de consumo en vatios y kilovatios-hora deriva de un modelo matemático estandarizado.

5.4. Auditoría de Implementación FURPS+ (Entregable 03)

La auditoría independiente documentada en docs/calidad/03_auditoria_implementacion_furps.md verificó la efectividad de la Ola 1:

- La pregunta M-U07 transitó de No cumple (1 punto) a Cumple (5 puntos).
- El puntaje bruto del factor Usabilidad ascendió de 32 a 36 puntos (incrementando su nivel de apego del 80.00 % al 90.00 %).
- El puntaje total bruto de FURPS+ se elevó de 168 / 200 (84.0 %) a 172 / 200 (86.0 %), lo que representa un valor ponderado de 84.78 a 86.28 / 100. Estas cifras incorporan el reajuste de M-R01 a Parcial (3) de la revisión del 12/11/2026; el documento 03 conserva los valores históricos 170 → 174.
- Se verificó la total ausencia de regresiones: la suite de pruebas del frontend ratificó 48 de 48 pruebas exitosas.

6. Fase 2 — Seguridad de Software (OWASP Top 10)

6.1. Auditoría Inicial de Seguridad OWASP (Entregable 04)

La evaluación de seguridad estática registrada en docs/calidad/04_auditoria_owasp.md analizó el perímetro web, los servicios de backend y las canalizaciones de CI/CD. Se determinó un Score Global de Seguridad inicial de 79.0 / 100 puntos, caracterizado por una defensa perimetral sólida pero con debilidades identificadas en configuraciones web y riesgos de cadena de suministro. La auditoría original (04) se elaboró con la lista OWASP Top 10:2021 (80.5 / 100) y fue reclasificada a la edición 2025 (79.0 / 100), según docs/calidad/08_reclasificacion_owasp_2025.md.

ID OWASP	Categoría de Riesgo	Estado Inicial	Severidad Máxima	Puntuación (0-10)	Principales Hallazgos Identificados
A01:2025	Broken Access Control (incluye SSRF)	Requiere Atención	Alto (verificado 12/11/2026)	7.0 / 10	Puertos de Docker Compose publicados (OW-02): los 5 solo escuchan en 127.0.0.1 (acceso local, no desde la red; severidad baja; evidencia S29). RLS permisivo verificado en Supabase (OW-01): 3 políticas «Permitir todo» ALL/true para public en hardware, logs y usuario (evidencia S11). SSRF (absorbida desde A10:2021) mitigada: rutas upstream fijas, rechazo de redirecciones en cliente HTTP y validación de esquemas URL.
A02:2025	Security Misconfiguration	Requiere Atención	Medio	7.0 / 10	Ausencia de cabeceras defensivas HTTP (CSP, X-Frame-Options) en Nginx (OW-06) y sobrescritura en location de configuración (OW-07); corregidos en Ola 1. Evidencia: S07.
A03:2025	Software Supply Chain Failures	Requiere Atención	Medio	8.0 / 10	Inclusión de scripts JS y hojas CSS externas desde CDN sin hash de integridad Subresource Integrity (OW-08; evidencia S08). Los hallazgos de CI/CD (OW-10, OW-11) se mantienen en A08 según el mapeo oficial, aunque A03:2025 también abarca la cadena CI/CD.
A04:2025	Cryptographic Failures	Requiere Atención	Alto (verificado 12/11/2026)	7.5 / 10	Columnas password y password_hash en la tabla usuario, verificadas en Supabase (OW-03; evidencia S12) y contraseñas sintéticas legibles en archivo Excel de datasets (OW-04; evidencia S30).
A05:2025	Injection	Conforme / Mitigado	Bajo	9.5 / 10	Consultas parametrizadas con SQLite, escape riguroso en PostgREST y listas blancas estrictas en PromQL. Evidencia: S37.
A06:2025	Insecure Design	Requiere Atención	Medio	8.0 / 10	Ausencia de Rate Limiting perimetral en Gateway para peticiones de inferencia pesadas (OW-05; evidencia S21).
A07:202	Authentication	Conforme	Medio	8.5 / 10	Verificación rigurosa de claims JWT en

ID OWASP	Categoría de Riesgo	Estado Inicial	Severidad Máxima	Puntuación (0-10)	Principales Hallazgos Identificados
5	Failures	con Observación			Gateway; token de sesión persistido en localStorage del navegador (OW-09; evidencia S22).
A08:2025	Software or Data Integrity Failures	Requiere Atención	Bajo	7.5 / 10	Acciones de GitHub CI referenciadas con tags mutables v4/v5 (OW-10; evidencia S09) e imágenes base de Docker sin fijación por digest SHA256 (OW-11; parcial: gateway-go ya fija @sha256 y el resto usa solo etiqueta; evidencia S31).
A09:2025	Security Logging and Alerting Failures	Requiere Atención	Medio	8.0 / 10	Correlación distribuida con X-Request-Id y 6 alertas Prometheus; sin registro estructurado de auditoría para eventos 401/403 en Gateway (OW-12; evidencia S23).
A10:2025	Mishandling of Exceptional Conditions	Requiere Atención	Medio	8.0 / 10	Fortalezas fail-closed: manejadores globales con respuestas 500 genéricas sin traza (Prediction, Data Processing, Monitoring), JWKS no disponible rechaza el token con 503, timeouts por ruta (504) y mensajes amigables en frontend. Brechas (OW-13): gateways sin manejo explícito de fallos propios (recover en Go, manejador global en Spring Cloud Gateway) y falta de pruebas que garanticen que los catch-all no filtran detalles internos. Evidencia: S24.
GLOBAL	**OWASP Top 10:2025**	**Nivel Bueno / Robusto**	**Medio Global**	**79.0 / 100**	**Perímetro seguro con recomendaciones focalizadas de remediación.**

6.2. Plan de Remediación OWASP (Entregable 05)

El documento docs/calidad/05_plan_owasp.md definió la hoja de ruta técnica para subsanar los 13 hallazgos de seguridad (OW-01 a OW-13), agrupándolos estratégicamente en tres olas:

- Ola 1 - Inmediata (Semana 13, 07–09/11/2026): Endurecimiento perimetral y cadena de suministro sin riesgo de regresión: incorporación de cabeceras HTTP de protección en Nginx (OW-06, OW-07), incorporación de hashes SRI en scripts y hojas de estilo de CDN (OW-08) y fijación inmutable de GitHub Actions mediante SHA (OW-10). Evidencia: OW-06 y OW-07 en S07, OW-08 en S08, OW-10 en S09.
- Ola 2 - Sprint 7 (17–23/11/2026): Seguridad en persistencia externa y Gateway: revisión y aplicación de políticas RLS restrictivas en Supabase (OW-01), retiro de la columna password en base de datos (OW-03), implementación de filtro de Rate Limiting por Token Bucket en Gateway (OW-05), registro estructurado de auditoría para eventos 401/403 (OW-12) y Content-Security-Policy mediante meta etiqueta HTML. Evidencia: OW-01 en S11, OW-03 en S12, OW-05 en S21, OW-12 en S23.
- Ola 3 - Sprint 8 (24–30/11/2026): Endurecimiento avanzado de contenedores y sesión: fijación de imágenes Docker base por digest SHA256 (OW-11), retiro de puertos internos en

compose (OW-02), depuración de contraseñas de archivo Excel (OW-04) y transición de almacenamiento de sesión de localStorage hacia cookies con atributos HttpOnly y SameSite (OW-09) y manejo explícito de condiciones excepcionales en los gateways, con pruebas de no filtración en los catch-all 500 (OW-13, A10:2025). Evidencia: OW-02 en S29, OW-04 en S30, OW-09 en S22, OW-11 en S31, OW-13 en S24.

6.3. Aplicación del Plan — Ola 1 de Seguridad

La ejecución de la Ola 1 de seguridad fue concretada y verificada mediante el commit d24d86f en la rama test, implementando las siguientes modificaciones. Commit verificable de la Ola 1: https://github.com/woshtsu/green-ai-workspace-cld/commit/d24d86f

1. OW-06 y OW-07: Blindaje de Cabeceras HTTP en Nginx En software/frontend/nginx.conf se incorporaron directivas de seguridad para mitigar ataques de Clickjacking, MIME-sniffing y fugas de contexto:
    - X-Frame-Options: DENY (impide que la consola sea incrustada en iframes maliciosos).
    - X-Content-Type-Options: nosniff (fuerza al navegador a respetar los tipos MIME declarados).
    - Referrer-Policy: no-referrer (evita el envío de URLs internas en peticiones salientes).
    - Permissions-Policy: geolocation=(), camera=(), microphone=() (deshabilita APIs invasivas del navegador).
    - Cross-Origin-Opener-Policy: same-origin (aísla el contexto de ejecución de ventanas secundarias). Asimismo, en el bloque específico location = /config.local.js se replicaron explícitamente estas directivas defensivas junto con Cache-Control: no-store, subsanando el comportamiento de Nginx que descarta cabeceras heredadas cuando un bloque hijo define encabezados propios.
2. OW-08: Subresource Integrity (SRI) en Recursos Externos Se añadieron atributos integrity con algoritmo SHA-384 y crossorigin="anonymous" en las 6 páginas HTML del frontend (index.html, login.html, register.html, dashboard.html, network.html y flows.html) para todas las bibliotecas consumidas desde CDN:
    - Supabase JS (v2.117.1): sha384-q36VSR1ktH2hgqgm+0LDeyqPW8CyZHSxfDSttSPm5ReOJzB+sYThmh7D4fE+jWTs.
    - Chart.js (v4.4.8): sha384-T/4KgSWuZEPozpPz7rnnp/5lDSnpY1VPJCojf1S81uTHS1E38qgLfMgVsAeRCWc4.
    - Font Awesome (v6.4.0): sha384-iw3OoTErCYJJB9mCa8LNS2hbsQ7M3C0EpIsO/H5+EGAkPGc6rk+V8i04oW/K5xq0.
      Esto inmuniza a los clientes frente al envenenamiento o alteración arbitraria de dependencias en servidores CDN externos.
3. OW-10: Inmunización de la Cadena de Suministro en CI/CD En los flujos .github/workflows/ci.yml y .github/workflows/pages.yml, se sustituyeron el 100 % de las referencias a GitHub Actions con etiquetas mutables (@v4, @v5) por sus respectivos hashes de commit inmutables de 40 caracteres (SHA-1), conservando la versión semántica como comentario inline (ej. actions/checkout@11d5960a326750d5838078e36cf38b85af677262 # v4).

6.4. Auditoría de Implementación OWASP (Entregable 06)

El informe docs/calidad/06_auditoria_implementacion_owasp.md constató la correcta aplicación de los cambios:

- Ninguna regresión detectada: La suite de frontend ejecutó 48 de 48 pruebas satisfactoriamente y la sintaxis de Nginx fue validada formalmente con resultado exitoso.
- Recálculo del Score Global de Seguridad: El puntaje ascendió de 79.0 / 100 a 83.5 / 100 (+4.5 puntos netos):
    - A02 (Security Misconfiguration): Pasa de 7.0 a 9.0 (+2.0 puntos) por cabeceras HTTP y corrección en config.local.js.
    - A03 (Software Supply Chain Failures): Pasa de 8.0 a 9.5 (+1.5 puntos) gracias a la implementación integral de hashes SRI.
    - A08 (Software or Data Integrity Failures): Pasa de 7.5 a 8.5 (+1.0 punto) por la fijación inmutable de GitHub Actions.
    - Las categorías A01, A04, A06, A07, A09 y A10 permanecen estables según su programación para las Olas 2 y 3.

7. Matriz de evaluación FURPS+ y OWASP aplicada al proyecto

La siguiente tabla refleja la evaluación técnica individual sobre las 50 preguntas que integran el marco de calidad de Green AI: las 40 preguntas del modelo FURPS+ y las 10 preguntas correspondientes al factor técnico de seguridad OWASP Top 10. La escala de cumplimiento docente corresponde a: Cumple (5 puntos), Parcial (3 puntos) y No cumple (1 punto).

7.1. Matriz de Evaluación Docente Completa (50 Preguntas)

FACTOR	id	PREGUNTA	ARTEFACTO	CUMPLIMIENTO	ACCIÓN CORRECTIVA O EVIDENCIA	RESPONSABLE	FECHA	PO	PD	NIVEL APEGO (%)
F	M-F01	¿Se ha implementado el caso de uso de visualización de métricas en tiempo real (HU-01)?	SOFTWARE	Cumple (5)	Catálogo y telemetría de 50 servidores vía Prometheus y Monitoring. Fuente: software/monitoring y dashboard.html. Smoke 14/14. Evidencia: S40	—	—	5	5	100.0 %
F	M-F02	¿Se ha implementado el caso de uso de consulta y paginación de históricos (HU-02)?	SOFTWARE	Cumple (5)	Endpoint /api/v1/historical-logs con paginación keyset y adaptador dual SQLite/Supabase. 114 tests de contrato. Evidencia: S41	—	—	5	5	100.0 %
F	M-F03	¿Se ha implementado el caso de uso de predicción de demanda de CPU a 15 min (HU-03)?	SOFTWARE	Cumple (5)	Inferencia con Random Forest vía POST /v1/predictions con metadatos y origin=estimated. 100 pruebas pytest. Evidencia: S48b	—	—	5	5	100.0 %
F	M-F04	¿Se ha implementado la comparación de alternativas de modelos y estrategias (HU-04)?	SOFTWARE	Cumple (5)	Comparación de 4 modelos en pipeline y experimento Baseline reactivo vs Green AI predictivo en UI. Fuente: plan experimental y dashboard. Evidencia: S43	—	—	5	5	100.0 %
F	M-F05	¿Se ha implementado el registro y visualización de experimentos en MLflow (HU-05)?	HERRAMIENTA	Parcial (3)	Corridas registradas localmente en software/prediction/mlruns con captura de UI local (§9.2); falta servidor público en la nube. Evidencia: C303 (§9.2)	Gina Barrera Peña	14/11/2026	3	5	60.0 %
F	M-F06	¿Se ha implementado el caso de uso de recomendación con restricciones de política (HU-06)?	SOFTWARE	Cumple (5)	Evaluación de CPU, memoria, latencia y errores con executionAllowed=false para control humano. Fuente: recommendations.py. Evidencia: S48c	—	—	5	5	100.0 %
F	M-F07	¿Se ha implementado la aprobación humana persistida de recomendaciones (HU-07)?	SOFTWARE	No cumple (1)	No existe flujo interactivo de aprobar/rechazar ni persistencia de decisiones en base de datos; solo bandera defensiva. Evidencia: S15	Ricardo Porras Veli	20/11/2026	1	5	20.0 %

FACTOR	id	PREGUNTA	ARTEFACTO	CUMPLIMIENTO	ACCIÓN CORRECTIVA O EVIDENCIA	RESPONSABLE	FECHA	PO	PD	NIVEL APEGO (%)
F	M-F08	¿Se ha implementado el cálculo de energía y CO2e a partir de potencia (HU-09)?	SOFTWARE	Parcial (3)	Cálculo de kWh por regla del trapecio implementado; potencia modelada sintéticamente, no de hardware real. Factor CO2e manual. Evidencia: S16	Gina Barrera Peña	21/11/2026	3	5	60.0 %
F	M-F09	¿Se ha implementado el dashboard integrado de observabilidad (HU-11)?	SOFTWARE	Parcial (3)	Implementado en software/frontend/dashboard.html; probado en local; pendiente verificación con sesión real de Supabase en /dev/. Evidencia: S01	Leonid Huamán Collazos	12/11/2026	3	5	60.0 %
F	M-F10	¿Se ha implementado el sistema de reglas de alerta de infraestructura (HU-12)?	SOFTWARE	Cumple (5)	4 reglas configuradas en software/infraestructura/prometheus/alerts.yml, validadas formalmente con promtool. Evidencia: C304	—	—	5	5	100.0 %
F	M-F11	¿Se ha implementado la generación y descarga de reportes históricos (HU-13)?	SOFTWARE	Parcial (3)	Endpoint genera resumen JSON y exportación CSV plano; exportación en formato PDF sigue pendiente. Evidencia: S17 y S43	Ricardo Porras Veli	18/11/2026	3	5	60.0 %
F	M-F12	¿Se ha implementado el control de acceso y autorización RBAC (HU-14)?	SOFTWARE	Cumple (5)	Gateway valida claims user_role para roles ADMIN, OPERATOR y VIEWER. Fuente: application.yaml y routes.go. Evidencia: prueba automática del Gateway (código), sin captura	—	—	5	5	100.0 %
F	M-F13	¿Se ha implementado el registro e inicio de sesión seguro (HU-15)?	HOSTING	Parcial (3)	Flujo PKCE y trigger SQL en supabase-auth-setup.sql; sin pruebas automatizadas con usuario real; RLS permisiva verificada (S11), pendiente de restringir. Evidencia: S13	Ricardo Porras Veli	11/11/2026	3	5	60.0 %
U	M-U01	¿La interfaz cuenta con navegación organizada en	SOFTWARE	Cumple (5)	Consola organizada en 5 pestañas temáticas, layout responsive y	—	—	5	5	100.0 %

FACTOR	id	PREGUNTA	ARTEFACTO	CUMPLIMIENTO	ACCIÓN CORRECTIVA O EVIDENCIA	RESPONSABLE	FECHA	PO	PD	NIVEL APEGO (%)
		pestañas y consistencia visual?			navegación común en software/frontend. Evidencia: S40 y S13					
U	M-U02	¿El sistema dispone de soporte para temas visuales (modo oscuro y claro)?	SOFTWARE	Cumple (5)	Conmutador accesible con persistencia en localStorage y variables CSS en appearance.js. Evidencia: S45 y S13	—	—	5	5	100.0 %
U	M-U03	¿Se cuenta con visualizaciones avanzadas de infraestructura (Racks con mapa de calor)?	SOFTWARE	Cumple (5)	Matriz interactiva de 50 servidores en 5 racks con umbrales semafóricos en network.html y 20 pruebas unitarias. Evidencia: S06	—	—	5	5	100.0 %
U	M-U04	¿El sistema provee ayuda contextual comprensible y textos explicativos (botones «?»)?	SOFTWARE	Cumple (5)	Módulo de ayuda contextual detallando qué es, uso, cálculo y límites experimentales en help-content.js. Evidencia: S46	—	—	5	5	100.0 %
U	M-U05	¿Se traducen los errores técnicos de la API a mensajes amigables para el usuario?	SOFTWARE	Cumple (5)	Clasificación de fallas en lenguaje humano con botones de acción sin códigos crípticos en friendly-errors.js. Evidencia: S45	—	—	5	5	100.0 %
U	M-U06	¿La interfaz respeta estándares de accesibilidad (semántica HTML, ARIA y contraste)?	SOFTWARE	Parcial (3)	Marcado semántico y atributos ARIA completos; pendiente escaneo formal WCAG 2.1 AA con Lighthouse y tabla para Chart.js. Evidencia: S20	Leonid Huamán Collazos	15/11/2026	3	5	60.0 %
U	M-U07	¿Se cuenta con una guía rápida de usuario para la operación del sistema?	GUÍA INST.	Cumple (5)	Subsanado en Ola 1: entregables/guia-de-uso/GUIA-RAPIDA-USUARIO.md documenta 8 flujos en <= 3 pasos con textos reales de la interfaz. Evidencia: S05	Jhonny Quintana Carhuaricra	09/11/2026 (hecha en la Ola 1)	5	5	100.0 %
U	M-U08	¿Se completa cualquier operación principal en tres	SOFTWARE	Parcial (3)	Operaciones comunes en <= 3 pasos; cálculo de energía con archivo y factor toma 4 pasos, e inferencia CPU usa	Leonid Huamán Collazos	17/11/2026	3	5	60.0 %

FACTOR	id	PREGUNTA	ARTEFACTO	CUMPLIMIENTO	ACCIÓN CORRECTIVA O EVIDENCIA	RESPONSABLE	FECHA	PO	PD	NIVEL APEGO (%)
		pasos o menos?			pestañas separadas. Evidencia: S48a, S48b y S48c					
R	M-R01	¿Se manejan los errores de las APIs mediante el estándar RFC 7807 (problem+json)?	SOFTWARE	Parcial (3)	Gateway Go, Data Processing y Monitoring responden application/problem+json; Prediction usa su propio sobre {code, message} (S28). Evidencia: S28	Ricardo Porras Veli	23/11/2026	3	5	60.0 %
R	M-R02	¿Todos los microservicios exponen sondas de salud (health checks / liveness / readiness)?	SOFTWARE	Cumple (5)	Sondas /health/ready, Actuator y binario -healthcheck en Go operando con normalidad. Evidencia: S28	—	—	5	5	100.0 %
R	M-R03	¿El sistema implementa timeouts defensivos para mitigar caídas de servicios upstream?	SOFTWARE	Cumple (5)	Timeouts escalonados en Gateway (6 s, 15 s, 30 s) y Data Processing (10 s) para evitar bloqueos en cascada. Evidencia: S32	—	—	5	5	100.0 %
R	M-R04	¿Se cuenta con una batería de pruebas automatizadas y smoke test end-to-end?	HERRAMIENTA	Cumple (5)	415+ pruebas automatizadas en 6 suites y smoke test de 14/14 pasos pasando satisfactoriamente. Evidencia: S10	—	—	5	5	100.0 %
R	M-R05	¿La plataforma resiste caídas y saturación sin comprometer la base de datos de producción?	HOSTING	Parcial (3)	SQLite aísla desarrollo local; falta verificar resistencia ante carga masiva en PostgREST remoto y restringir la RLS permisiva de Supabase (verificada, S11). Evidencia: S18	Ricardo Porras Veli	13/11/2026	3	5	60.0 %
R	M-R06	¿Se controla el retorno exitoso y no exitoso desde la base de datos?	SOFTWARE	Cumple (5)	Data Processing intercepta respuestas/errores de PostgREST y SQLite traduciéndolos a problem+json sin filtrar datos sensibles. Evidencia: S33	—	—	5	5	100.0 %
P	M-P01	¿La operación y consulta responde en promedio en	SOFTWARE	Parcial (3)	Local responde en < 30 ms (Java) y < 4 ms (Go); en Render Free Tier el cold	Leonid Huamán	19/11/2026	3	5	60.0 %

FACTOR	id	PREGUNTA	ARTEFACTO	CUMPLIMIENTO	ACCIÓN CORRECTIVA O EVIDENCIA	RESPONSABLE	FECHA	P O	P D	NIVEL APEGO (%)
		menos de 7 segundos hacia el usuario?			start toma ~60 s tras 15 min de inactividad. Evidencia: S25	Collazos				
P	M-P02	¿Hay un uso razonable del consumo de memoria RAM (< 512 MB por contenedor)?	SISTEMA OP.	Cumple (5)	Prediction consume 122–142 MiB; Gateway Go 2.3–9.8 MiB; Data Processing 76 MiB; Monitoring ~211 MiB. Evidencia: S34	—	—	5	5	100.0 %
P	M-P03	¿Se cuenta con imágenes de contenedor optimizadas y ligeras para inferencia?	HERRAMIENTA	Cumple (5)	Dockerfile multi-etapa en Prediction redujo imagen en 82 % (de 2.85 GB a 510 MB); Gateway Go distroless de 16.2 MB. Evidencia: S34	—	—	5	5	100.0 %
S	M-S01	¿El código fuente sigue los principios de arquitectura hexagonal (puertos y adaptadores)?	SOFTWARE	Parcial (3)	Hexagonal pura en backend; Simulator acopla adaptadores en application/runner.py importándolos directamente. Evidencia: S19	Ricardo Porras Veli	23/11/2026	3	5	60.0 %
S	M-S02	¿Se cuenta con especificaciones de contratos de interfaz OpenAPI versionadas?	SOFTWARE	Cumple (5)	Contratos OpenAPI versionados en Gateway, Monitoring, Data Processing y Swagger en Prediction. Evidencia: S26	—	—	5	5	100.0 %
S	M-S03	¿Se dispone de un flujo de integración continua (CI) que valide pruebas en cada cambio?	HERRAMIENTA	Cumple (5)	Workflow de GitHub Actions que ejecuta suites completas de pruebas en main y test. ci.yml y pages.yml blindados. Evidencia: S09	—	—	5	5	100.0 %
S	M-S04	¿El monorepo cuenta con estructura limpia sin repositorios Git anidados ni gitlinks?	HERRAMIENTA	Cumple (5)	Repositorio unificado en la raíz; verificación git ls-files -s sin modo 160000; ramas principales main y test. Evidencia: S26	—	—	5	5	100.0 %
PLUS	M-PL01	¿Cuenta con la guía de instalación y despliegue paso	GUÍA INST.	Cumple (5)	Guía detallada en docs/GUIA-DESPLIEGUE-RENDER.md para desplegar en Render y GitHub Pages sin secretos	—	—	5	5	100.0 %

FACTOR	id	PREGUNTA	ARTEFACTO	CUMPLIMIENTO	ACCIÓN CORRECTIVA O EVIDENCIA	RESPONSABLE	FECHA	PO	PD	NIVEL APEGO (%)
		a paso para la nube?			versionados. Evidencia: S27					
PLUS	M-PL02	¿Se cuenta con un instalador o mecanismo automático de arranque local (docker-compose)?	HERRAMIENTA	Cumple (5)	Archivo Docker Compose multi-servicio que arranca el stack completo con docker compose up en software/infraestructura. Evidencia: S27	—	—	5	5	100.0 %
PLUS	M-PL03	¿Se cuenta con los espacios para alojar el software en calidad a costo cero?	HOSTING	Parcial (3)	Frontend en Pages y backend en Render Free Tier; limitaciones de suspensión (15 min) y cuota de 750 h/mes compartida. Evidencia: S14	Leonid Huamán Collazos	22/11/2026	3	5	60.0 %
PLUS	M-PL04	¿El sistema invoca a otros sistemas externos de manera desacoplada y segura?	SOFTWARE	Cumple (5)	Conexión segura a Supabase Auth por JWKS público; sin llaves service_role en frontend ni backend. Las APIs de datos pasan por el Gateway; el inicio de sesión va directo a Supabase Auth. Evidencia: S35	—	—	5	5	100.0 %
PLUS	M-PL05	¿El proyecto cumple con las restricciones de licencias de software y normativa legal?	SOFTWARE	Cumple (5)	Dependencias Open Source compatibles (Apache 2.0 / MIT); apego explícito a la Ley N° 29733 en el frontend. Licencia MIT del proyecto en LICENSE (raíz del repositorio). Evidencia: S36	—	—	5	5	100.0 %
PLUS	M-PL06	¿Se cuenta con una guía de inicio o arranque del sistema?	GUÍA INST.	Cumple (5)	Documentado en README.md:L22-L38 y docs/CONTINUAR-EN-NUEVO-CHAT.md para Docker Compose, puertos y ejecución de pruebas. Evidencia: S27	—	—	5	5	100.0 %
OWASP	OW-A01	¿Se tiene implementado el control de acceso contra A01:2025: Broken Access Control (incluye SSRF)?	SOFTWARE	No cumple (1)	RBAC sólido en Gateway (ADMIN/OPERATOR/VIEWER) y SSRF mitigado (rutas upstream fijas, validación de esquemas URL, rechazo de redirecciones); RLS verificada como	Ricardo Porras Veli (con el dueño de Supabase)	23/11/2026	1	5	20.0 %

FACTOR	id	PREGUNTA	ARTEFACTO	CUMPLIMIENTO	ACCIÓN CORRECTIVA O EVIDENCIA	RESPONSABLE	FECHA	PO	PD	NIVEL APEGO (%)
					permisiva en Supabase (3 políticas «Permitir todo»); pendiente restringirla y acotar los puertos locales de compose (solo 127.0.0.1). Evidencia: S11 y S29					
OWASP	OW-A02	¿Se tiene implementada la configuración segura contra A02:2025: Security Misconfiguration?	SOFTWARE	Cumple (5)	Subsanado en Ola 1: cabeceras defensivas (X-Frame-Options, nosniff, COOP) en nginx.conf y corrección en config.local.js. Evidencia: S07	Ricardo Porras Veli	07/11/2026	5	5	100.0 %
OWASP	OW-A03	¿Se tiene implementado el control contra A03:2025: Software Supply Chain Failures?	SOFTWARE	Cumple (5)	Subsanado en Ola 1: hashes Subresource Integrity (SRI) y crossorigin en todas las páginas HTML para scripts CDN. Evidencia: S08	Ricardo Porras Veli	07/11/2026	5	5	100.0 %
OWASP	OW-A04	¿Se tiene implementada la protección criptográfica contra A04:2025: Cryptographic Failures?	SOFTWARE	No cumple (1)	TLS forzado y verificación JWT asimétrica; verificadas en Supabase las columnas password y password_hash en usuario; pendiente eliminar la columna en BD y las contraseñas del dataset Excel. Evidencia: S12 y S30	Gina Barrera Peña (con el dueño de Supabase)	30/11/2026	1	5	20.0 %
OWASP	OW-A05	¿Se tiene implementada la prevención contra A05:2025: Injection?	SOFTWARE	Cumple (5)	Parámetros ligados en SQLite (?), escape en PostgREST, lista blanca PromQL y modelos Pydantic restrictivos. Evidencia: S37	—	—	5	5	100.0 %
OWASP	OW-A06	¿Se tiene implementado el diseño seguro contra A06:2025: Insecure Design?	SOFTWARE	Parcial (3)	Arquitectura hexagonal y principio fail-secure en recomendaciones; pendiente Rate Limiting perimetral en Gateway. Evidencia: S21	Ricardo Porras Veli	23/11/2026	3	5	60.0 %
OWASP	OW-A07	¿Se tiene implementada la autenticación robusta contra A07:2025: Authentication	SOFTWARE	Parcial (3)	Validación exhaustiva de tokens JWT y claims en Gateway; observación por persistencia de token en localStorage.	Leonid Huamán Collazos	30/11/2026	3	5	60.0 %

FACTOR	id	PREGUNTA	ARTEFACTO	CUMPLIMIENTO	ACCIÓN CORRECTIVA O EVIDENCIA	RESPONSABLE	FECHA	PO	PD	NIVEL APEGO (%)
		Failures?			Evidencia: S22					
OWASP	OW-A08	¿Se tiene implementada la integridad de software contra A08:2025: Software or Data Integrity Failures?	HERRAMIENTA	Cumple (5)	Subsanado en Ola 1: anclaje inmutable del 100 % de GitHub Actions por commit SHA en ci.yml y pages.yml. Imágenes base Docker (OW-11) parcial: gateway-go ya fija @sha256; el resto usa solo etiqueta. Evidencia: S09 y S31	Ricardo Porras Veli	07/11/2026	5	5	100.0 %
OWASP	OW-A09	¿Se tiene implementado el registro y las alertas contra A09:2025: Security Logging and Alerting Failures?	SOFTWARE	Parcial (3)	Trazabilidad con X-Request-Id y alertas Prometheus; pendiente registro de auditoría estructurado para 401/403 en Gateway. Evidencia: S23	Ricardo Porras Veli	23/11/2026	3	5	60.0 %
OWASP	OW-A10	¿Se tiene implementado el manejo seguro de condiciones excepcionales contra A10:2025: Mishandling of Exceptional Conditions?	SOFTWARE	Parcial (3)	Fortalezas fail-closed: catch-all 500 genéricos sin traza, JWKS no disponible rechaza el token (503), timeouts por ruta y mensajes amigables en frontend; pendiente OW-13: recover en gateway-go, manejador global en Spring Cloud Gateway y pruebas de no filtración. Evidencia: S24	Ricardo Porras Veli	30/11/2026	3	5	60.0 %

7.2. Tabla de Nivel de Apego por Factor (Inicial, Actual y Esperado)

Los puntajes consolidados del proyecto se resumen a continuación, reflejando el progreso obtenido tras la ejecución de la Ola 1 y la proyección hacia el cierre del ciclo de vida del proyecto. Los valores corresponden fielmente a los registros del archivo oficial exportados/semana13/MODELO-FURPSPLUS-Green-AI.xlsx generado para la sesión:

Factor Evaluado	Preguntas	Puntaje Deseado (PD)	Puntaje Inicial (PO)	Apego Inicial (%)	Puntaje Actual Post Ola 1	Apego Actual (%)	Puntaje Esperado (Meta)	Apego Esperado (%)	Variación Neta Ola 1
F — Funcionalidad	13	65	51	78.5 %	51	78.5 %	63	96.9 %	0.0 %
U — Usabilidad	8	40	32	80.0 %	36	90.0 %	40	100.0 %	+10.0 %
R — Confiabilidad	6	30	26	86.7 %	26	86.7 %	30	100.0 %	0.0 %
P — Rendimiento	3	15	13	86.7 %	13	86.7 %	15	100.0 %	0.0 %
S — Soporte / Mantenibilidad	4	20	18	90.0 %	18	90.0 %	20	100.0 %	0.0 %
+ — Restricciones Adicionales	6	30	28	93.3 %	28	93.3 %	28	93.3 %	0.0 %
SUBTOTAL FURPS+ (01–03)	**40**	**200**	**168**	**84.0 %**	**172**	**86.0 %**	**196**	**98.0 %**	**+2.0 %**
OWASP — Seguridad Web (04–06)	10	50	28	56.0 %	34	68.0 %	48	96.0 %	+12.0 %
TOTAL GENERAL CONSOLIDADO	**50**	**250**	**196**	**78.4 %**	**206**	**82.4 %**	**244**	**97.6 %**	**+4.0 %**

Nota metodológica sobre las métricas: 1. El informe docs/calidad/01_auditoria_furps.md reportó inicialmente un nivel de apego del 85.00 % (170 / 200), reajustado en la revisión del equipo del 12/11/2026 a 84.0 % (168 / 200) al pasar M-R01 a Parcial (3); evalúa de forma estricta las 40 preguntas del estándar FURPS+. Al integrar las 10 preguntas de seguridad OWASP Top 10 en la matriz consolidada docente de 50 ítems, el puntaje inicial total fue de 196 sobre 250 puntos, lo que arroja un nivel de calidad inicial global del 78.4 %. 2. Tras la ejecución de la Ola 1, el puntaje global ascendió a 206 sobre 250 puntos (82.4 %), impulsado por la subsanación de M-U07 en Usabilidad (+4 puntos brutos) y las remediaciones de seguridad en cabeceras Nginx, SRI y anclaje SHA de GitHub Actions (+6 puntos brutos en los factores OW-A02, OW-A03 y OW-A08). 3. El detalle exhaustivo fila por fila para cada uno de los tres escenarios se encuentra particionado en las hojas correspondientes («Inicial», «Actual» y «Esperado») dentro del libro de trabajo Excel adjunto MODELO-FURPSPLUS-Green-AI.xlsx. 4. La parte OWASP de la matriz fue reclasificada a la edición Top 10:2025 (SSRF absorbida en A01 y nueva categoría A10:2025 valorada como Parcial). Las preguntas OW-A01 (A01:2025) y OW-A04 (A04:2025) figuran como No cumple (1), igual que en el Excel oficial; la versión previa de este documento las mostraba por error como Parcial (3), lo que era incoherente con sus propios totales. Con la lista 2021 los totales eran 198 / 79.2 % (inicial) y 208 / 83.2 % (actual) tras el reajuste de M-R01.

8. Nivel de calidad inicial y esperado: análisis de brechas

La progresión desde un nivel de calidad inicial del 78.4 % hasta un nivel actual del 82.4 % y una meta proyectada del 97.6 % ilustra un avance metodológico sustentado en evidencias. Sin embargo, un principio central de la ingeniería de software es reconocer con honestidad técnica por qué el sistema no alcanza —ni debe pretender alcanzar en esta fase académica— un 100.0 % de cumplimiento perfecto.

El análisis de brechas identifica tres condicionantes estructurales que impiden alcanzar el puntaje ideal:

1. Condicionantes Físicas y Financieras del Hosting Gratuito (Render Free Tier): El despliegue de backend a costo cero constituye un logro significativo para el proyecto, pero introduce limitaciones intrínsecas:
    - Suspensión por inactividad: Render suspende los contenedores tras 15 minutos sin tráfico entrante. El encendido subsecuente (cold start) impone latencias de 45 a 70 segundos para la primera petición, lo que vulnera el criterio estricto de respuesta menor a 7 segundos (M-P01).
    - Límite de memoria y CPU: Cada servicio dispone de 512 MB de memoria RAM no ampliable y cuota compartida de 750 horas mensuales por cuenta de usuario (PLUS-02 / M-PL03).
    - Debido a que estas restricciones son inherentes al modelo de alojamiento sin costo, la pregunta M-PL03 se mantiene deliberadamente en Parcial (3 puntos, 28/30 en el factor «+»), impidiendo el 100 % en dicha categoría.
2. Honestidad Experimental en la Medición de Telemetría y Potencia Eléctrica: En la pregunta M-F08 (cálculo de energía y CO2e), el sistema aplica formalmente la regla del trapecio sobre valores de potencia en vatios. Sin embargo, la potencia no proviene de mediciones de hardware reales ni sensores IPMI/PDU físicos, sino de un modelo lineal aproximado P = P_idle + (P_max - P_idle) * CPU (software/frontend/energy.js). Simular que esta potencia es una medición de laboratorio real violaría las directrices de integridad científica de la universidad. Por ende, la pregunta M-F08 se califica en Parcial (3 puntos) y se mantendrá en ese estado hasta que el centro de datos cuente con instrumentación de hardware real conectada por protocolos SNMP o Redfish.
3. Desacoplamiento de Persistencia Externa en la Nube: En el factor de seguridad OWASP (OW-A01 y OW-A04), persisten hallazgos sobre la base de datos externa en Supabase, que ya se verificaron en la consola de Supabase el 12/11/2026: 3 políticas «Permitir todo» (ALL/true para public) en hardware, logs y usuario (S11) y las columnas password y password_hash en usuario (S12). Por eso la calificación se mantiene en No cumple (no en Parcial) hasta que se restrinjan las políticas y se retire la columna.

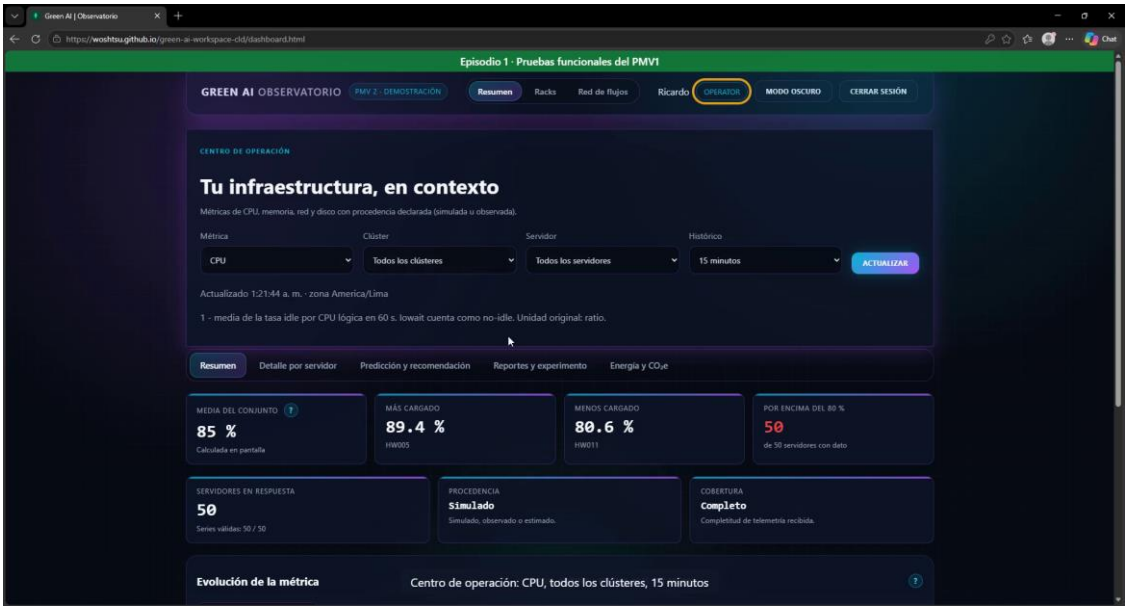
Estas brechas no constituyen negligencia técnica, sino la delimitación consciente entre el alcance demostrativo de un prototipo de ingeniería riguroso y la operación corporativa a escala productiva.

9. Evidencias técnicas por pregunta y verificación de la Ola 1

Para fundamentar las calificaciones de la matriz docente y constatar la efectividad de las mejoras implementadas en la Ola 1, se presentan a continuación los registros de ejecución, comandos y salidas reales obtenidos del entorno de verificación (docs/evidencias/semana13-ola1-verificacion.md), complementados con las referencias de capturas de pantalla requeridas.

9.1. Pregunta M-F09 (Dashboard Integrado de Observabilidad - HU-11)

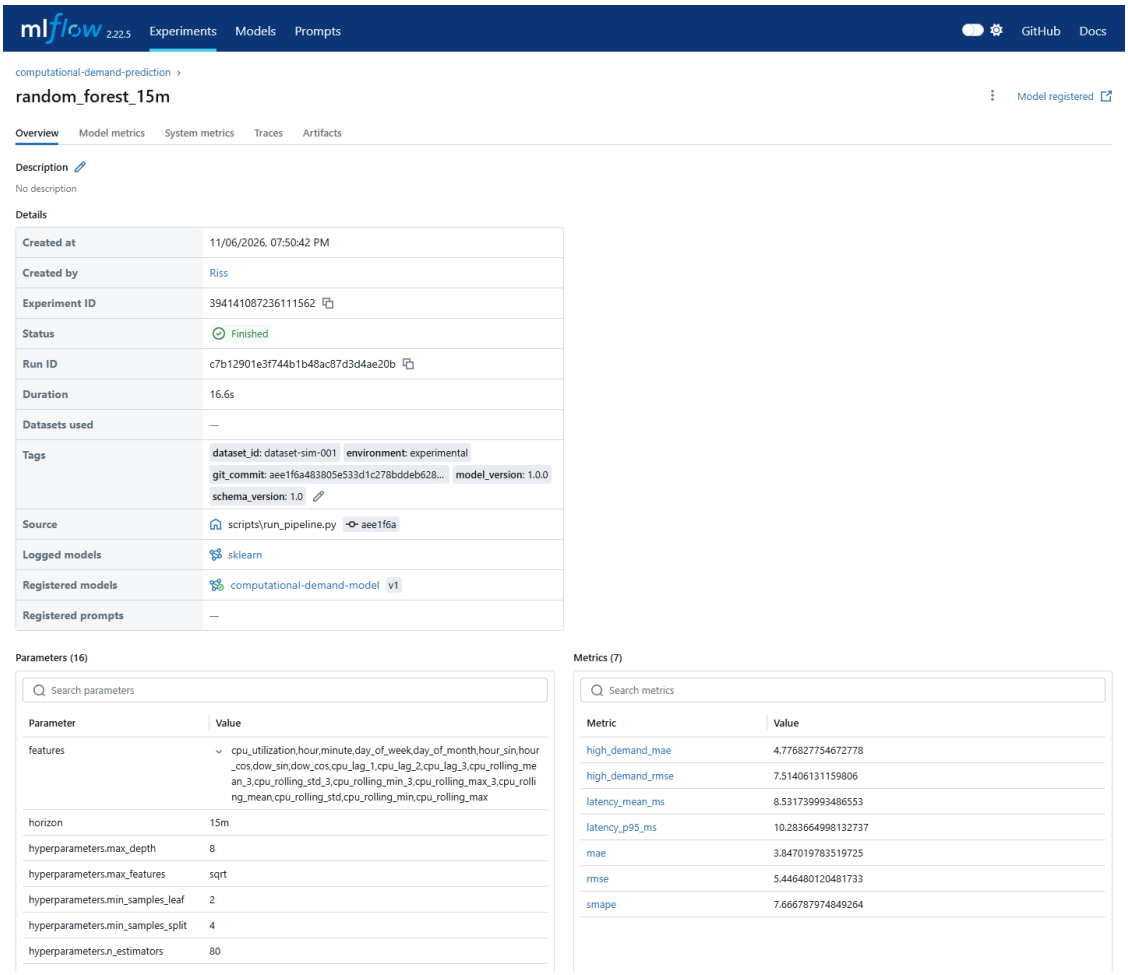
El Observatorio integra en una consola visual única el monitoreo de métricas en tiempo real, el detalle por servidor, la inferencia de demanda y la emisión de recomendaciones operativas.



Dashboard en producción con sesión de rol OPERATOR, las 5 pestañas y los KPI de CPU (fotograma del video de la Unidad III grabado el 10/11/2026, episodio 1)

9.2. Pregunta M-F05 (Registro y Visualización en MLflow - HU-05)

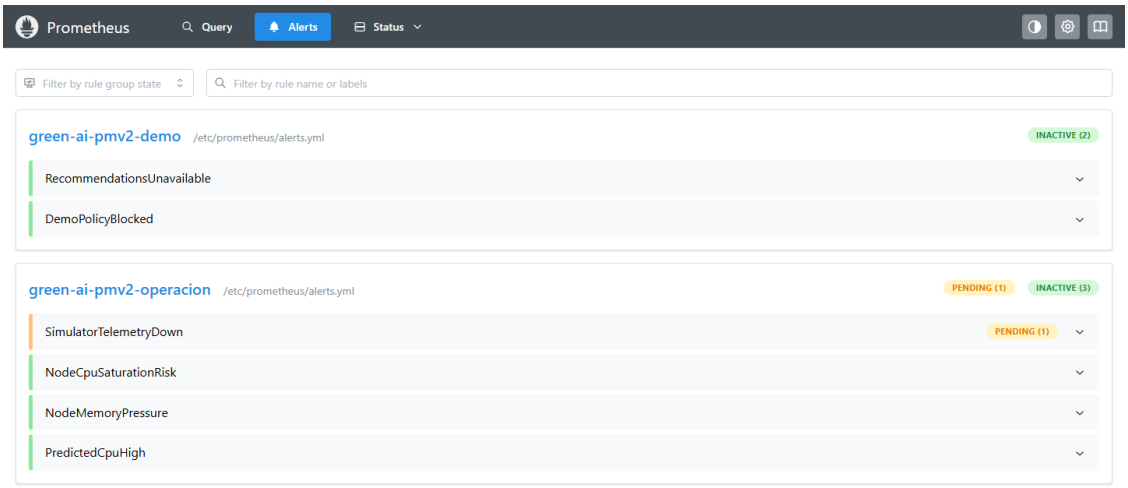
El entrenamiento y selección del modelo Random Forest para inferencia de CPU a 15 minutos se gestiona mediante MLflow, registrando parámetros de hiperparámetros, métricas de error (RMSE, MAE) y artefactos serializados.



Interfaz web de MLflow UI (v2.22.5) con la corrida real random_forest_15m del experimento computational-demand-prediction (Run ID c7b12901e3f744b1b48ac87d3d4ae20b, dataset-sim-001, RMSE = 5.45, MAE = 3.85)

9.3. Pregunta M-F10 (Sistema de Reglas de Alerta - HU-12)

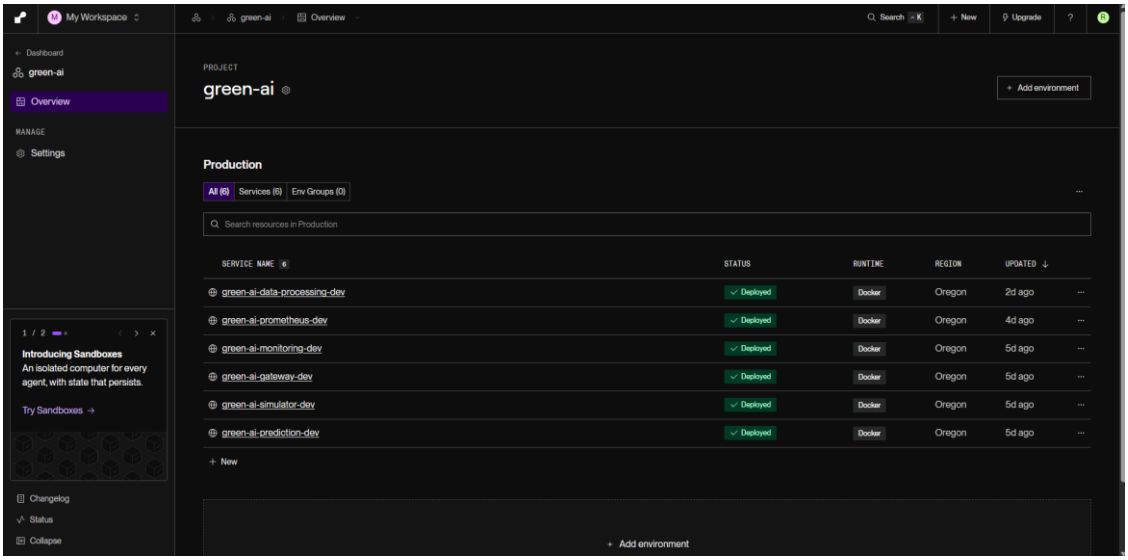
Prometheus supervisa continuamente los umbrales de infraestructura mediante cuatro reglas declarativas de operación (grupo `green-ai-pmv2-operacion`, más dos de demostración en `green-ai-pmv2-demo`) validadas con la herramienta promtool.



Capturar la consola web de Prometheus en la sección /alerts mostrando las reglas cargadas (SimulatorTelemetryDown, NodeC

9.4. Pregunta M-PL03 (Alojamiento en Hosting a Costo Cero)

La infraestructura de servicios contenerizados opera sobre el plan gratuito de Render, coordinando el API Gateway y los microservicios de backend.



Panel de Render, proyecto green-ai, entorno Production: los 6 servicios green-ai--dev en estado Deployed (Docker, Oregon)*

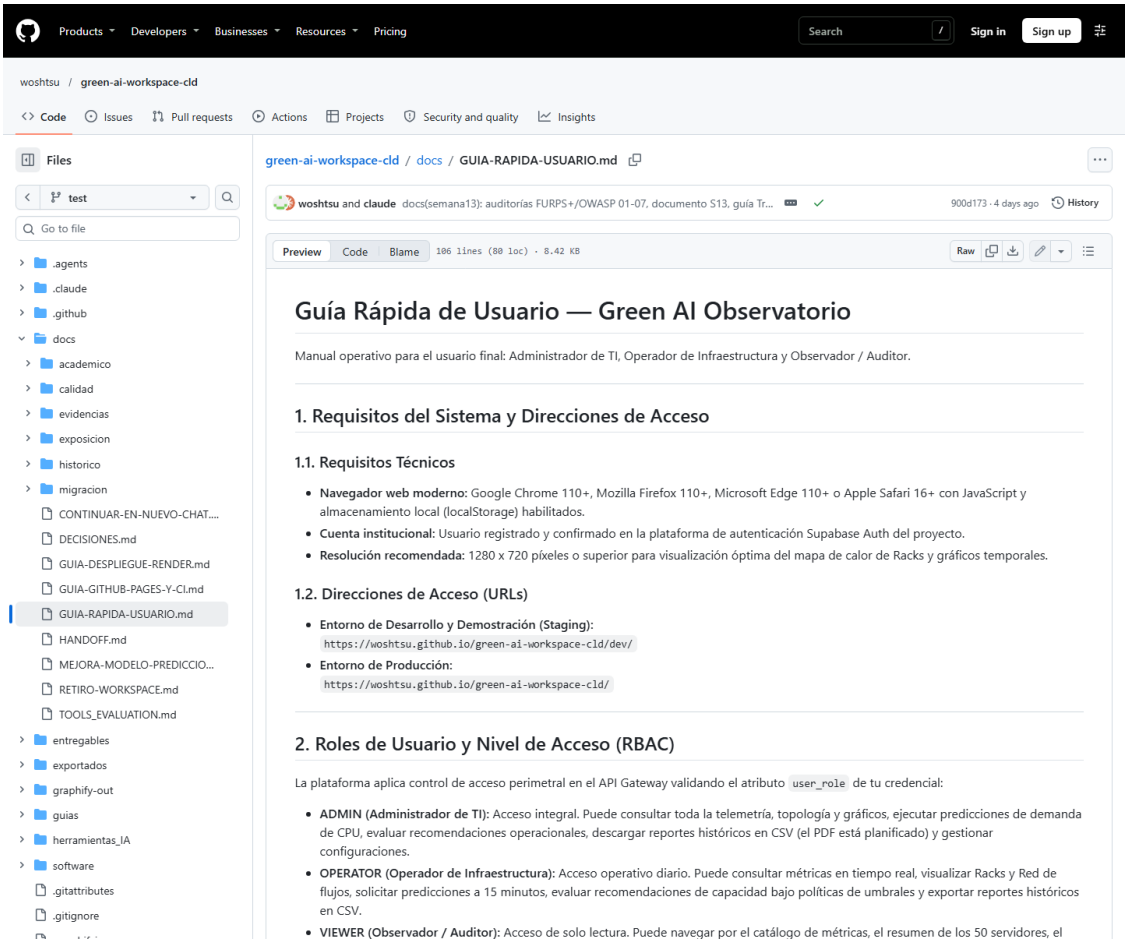
9.5. Pregunta M-U07 (Guía Rápida de Usuario — Mejora Ola 1)

Subsanación documental que estructura la interacción del usuario final en 8 tareas principales de 3 pasos o menos. La captura se tomó cuando la guía estaba en docs/GUIA-RAPIDA-USUARIO.md; desde el 11/11/2026 está en entregables/guia-de-uso/GUIA-RAPIDA-USUARIO.md.

```
# Verificación de correspondencia de textos reales entre la guía y la interfaz
grep -o -E "Registrar operador|Entrar al sistema|Centro de operación|Predecir 15
min|Evaluar recomendación" \
  software/frontend/*.html software/frontend/*.js
```

Resultado obtenido:

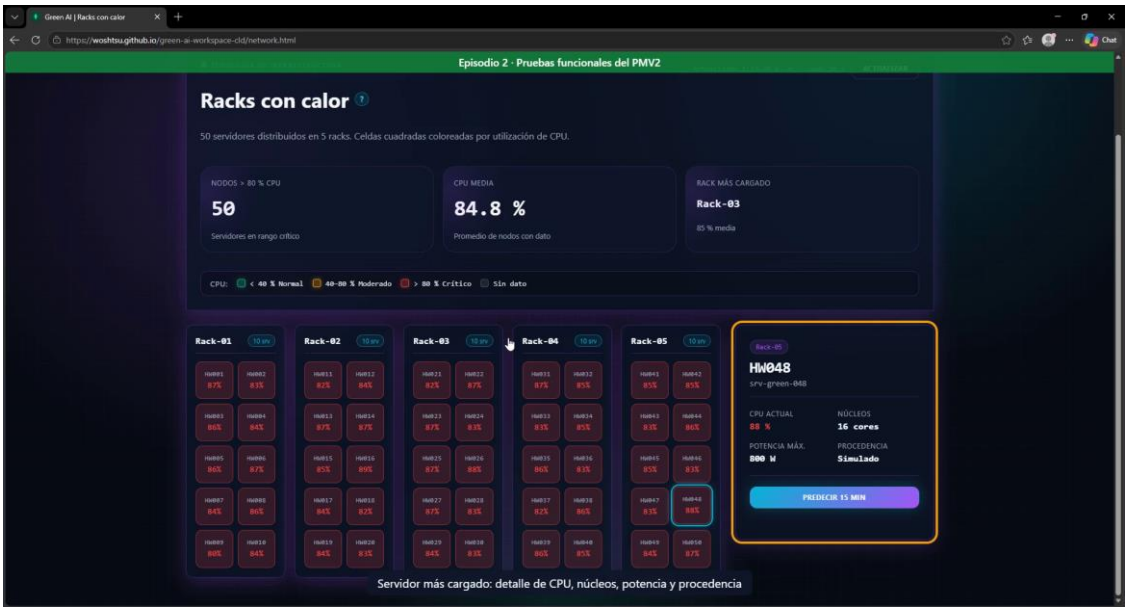
```
software/frontend/dashboard.html:Centro de operación
software/frontend/dashboard.html:Evaluar recomendación
software/frontend/login.html:Entrar al sistema
software/frontend/network.html:Predecir 15 min
software/frontend/register.html:Registrar operador
```



Capturar el documento entregables/guia-de-uso/GUIA-RAPIDA-USUARIO.md abierto y renderizado en el navegador o visor markd

9.6. Pregunta M-U03 (Visualización de Racks con Mapa de Calor)

Consola topológica interactiva que distribuye los 50 servidores simulados en 5 racks con código semafórico de carga de CPU.



Racks con calor: 50 servidores en 5 racks y panel lateral del servidor HW048 seleccionado (fotograma del video de la Unidad III grabado el 10/11/2026, episodio 2)

9.7. Pregunta OW-A02 / M-U01 / M-PL04 (Cabeceras Defensivas HTTP en Nginx — OW-06 y OW-07)

Verificación de la sintaxis y de las cabeceras servidas por el contenedor web Nginx:

```
# Verificación de sintaxis de Nginx
docker run --rm -v "$PWD/software/frontend/nginx.conf:/etc/nginx/conf.d/default.conf:ro"
nginx:1.28.0-alpine nginx -t
```

Resultado obtenido:

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

```
# Inspección de cabeceras en /login.html y /config.local.js
curl -sSI http://127.0.0.1:13099/config.local.js   # contenedor nginx:1.28.0-alpine con
nginx.conf montado
```

Resultado obtenido (07/11/2026; se omiten Date, ETag y Content-Length):

```
HTTP/1.1 200 OK
Server: nginx/1.28.0
Content-Type: application/javascript
Cache-Control: no-store
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
X-Frame-Options: DENY
Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=()
Cross-Origin-Opener-Policy: same-origin
```

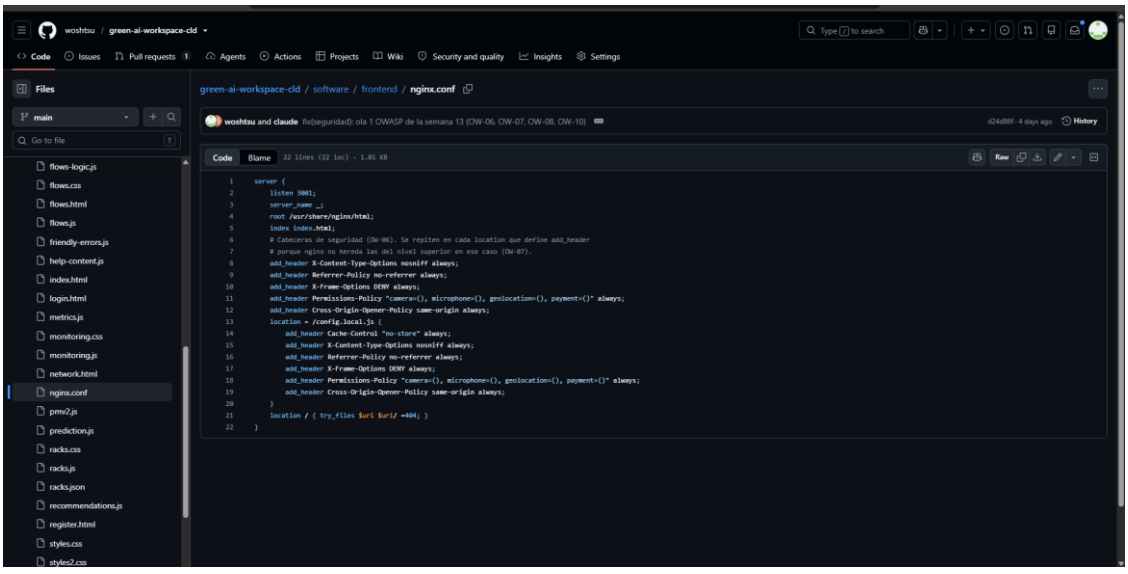
Verificación en ejecución aportada por Leonid Huamán (12/11/2026, 22:58 GMT), con las rutas relativas a la raíz del repositorio:

```
docker run --rm -d -p 8088:3001 -v
"$PWD/software/frontend/nginx.conf:/etc/nginx/conf.d/default.conf:ro" -v
"$PWD/software/frontend:/usr/share/nginx/html:ro" nginx:1.28.0-alpine
curl -sSI http://127.0.0.1:8088/login.html
```

```
HTTP/1.1 200 OK
Server: nginx/1.28.0
Date: Thu, 12 Nov 2026 22:58:48 GMT
Content-Type: text/html
Content-Length: 3048
Last-Modified: Thu, 12 Nov 2026 22:56:28 GMT
Connection: keep-alive
ETag: "6af6441c-be8"
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
X-Frame-Options: DENY
Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=()
Cross-Origin-Opener-Policy: same-origin
Accept-Ranges: bytes

curl -sSI http://127.0.0.1:8088/config.local.js
HTTP/1.1 404 Not Found
Server: nginx/1.28.0
Date: Thu, 12 Nov 2026 22:59:25 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive
Cache-Control: no-store
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
X-Frame-Options: DENY
Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=()
Cross-Origin-Opener-Policy: same-origin
```

El 404 de config.local.js es esperado (archivo local no versionado) y aun así lleva Cache-Control: no-store y las cinco cabeceras (OW-07 corregido).



Archivo software/frontend/nginx.conf en GitHub (rama main) con las cabeceras de seguridad OW-06 y su repetición en location = /config.local.js (OW-07); evidencia de configuración, no de ejecución

9.8. Pregunta OW-A03 (Subresource Integrity en CDN — OW-08)

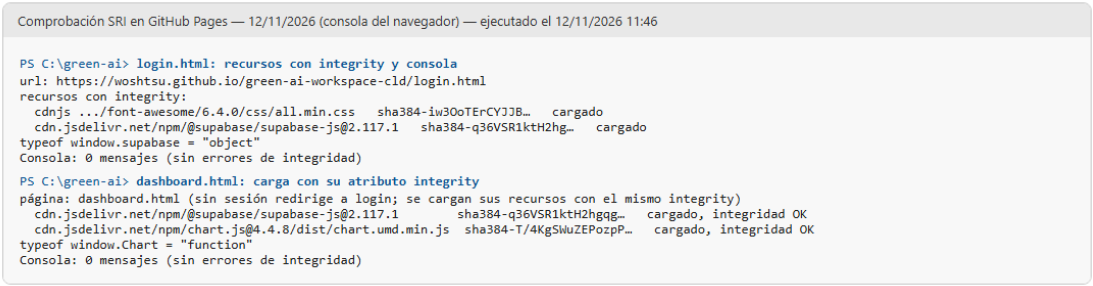
Comprobación de la estabilidad y consistencia de los hashes criptográficos generados:

```
# Verificación de hash SRI para supabase-js 2.117.1
curl -sSfL https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2.117.1 | \
  openssl dgst -sha384 -binary | openssl base64 -A
```

Resultado obtenido:

```
q36VSR1ktH2hgqgm+0LDeyqPW8CyZHSxfDSttSPm5ReOJzB+sYThmh7D4fE+jWTs
```

Verificación en navegador sin errores de integridad:



Comprobación SRI del 12/11/2026 en GitHub Pages: login.html y los recursos de dashboard.html cargan con su atributo integrity, window.supabase y window.Chart definidos, consola sin errores

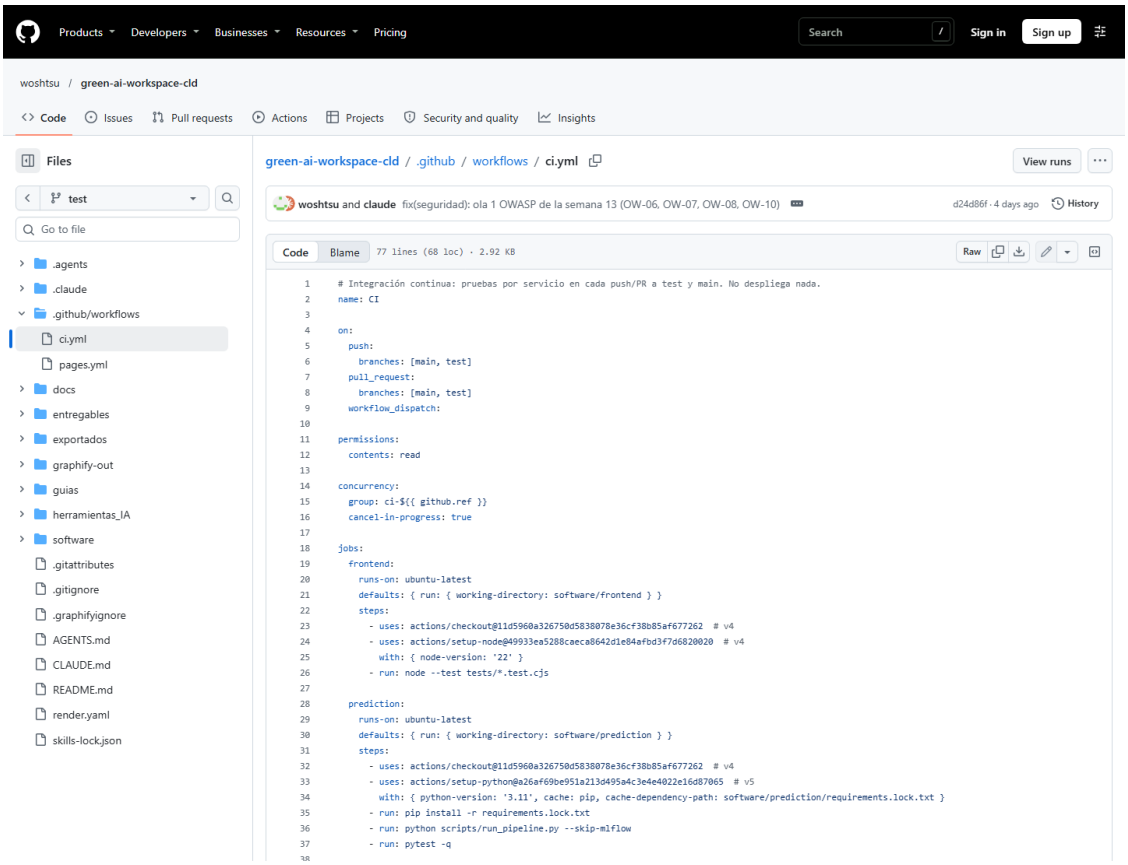
9.9. Pregunta OW-A08 (A08:2025) / M-S03 (Anclaje Inmutable de GitHub Actions — OW-10)

Inspección de las canalizaciones automatizadas en .github/workflows/ci.yml y pages.yml:

```
# Inspección de referencias inmutables SHA en ci.yml
grep -n "uses: actions/" .github/workflows/ci.yml
```

Resultado obtenido:

```
23:      - uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262  # v4
24:      - uses: actions/setup-node@49933ea5288caeca8642d1e84afbd3f7d6820020  # v4
32:      - uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262  # v4
33:      - uses: actions/setup-python@a26af69be951a213d495a4c3e4e4022e16d87065  # v5
43:      - uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262  # v4
44:      - uses: actions/setup-python@a26af69be951a213d495a4c3e4e4022e16d87065  # v5
53:      - uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262  # v4
54:      - uses: actions/setup-go@40f1582b2485089dde7abd97c1529aa768e1baff  # v5
63:      - uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262  # v4
64:      - uses: actions/setup-java@cf277c60eb25467037889841efdb72551f06f6c3  # v4
72:      - uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262  # v4
```



Capturar el código del archivo .github/workflows/ci.yml en el editor o repositorio mostrando las acciones fijadas por co

9.10. Verificación de Regresiones en Frontend (M-R04)

Ejecución de la suite modular de pruebas unitarias sobre el frontend tras las modificaciones de seguridad:

```
cd software/frontend && node --test tests/*.test.cjs
```

Resultado obtenido:

```
ℹ tests 48
ℹ suites 0
ℹ pass 48
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 496.619
```



Capturar la ventana de terminal con la salida exitosa del comando de pruebas de frontend mostrando los 48 tests pasados

10. Auditoría final integral: dictamen y recomendaciones

La síntesis de todo el ciclo de evaluación y mejora quedó formalizada en el informe docs/calidad/07_auditoria_final.md, emitido por el Auditor Principal de Calidad y Seguridad.

10.1. Dictamen Oficial de Calidad y Seguridad

El dictamen técnico emitido para el proyecto Green AI es:

APTO PARA PRODUCCIÓN CON OBSERVACIONES

10.2. Justificación Técnica del Dictamen

1. Cumplimiento de Calidad (FURPS+): El sistema alcanzó un nivel de apego auditado del **86.0 %** (86.28 % ponderado) en FURPS+, superando con holgura los requerimientos exigidos para un Producto Mínimo Viable (PMV2) de demostración académica.
2. Cumplimiento de Seguridad (OWASP Top 10:2025): El Score de Seguridad auditado post Ola 1 se situó en **83.5 / 100 (OWASP Top 10:2025)**, evidenciando un perímetro protegido contra falsificación de orígenes (CORS), inyecciones y adulteración de dependencias.
3. Preservación de Funcionalidad: Cero (0) regresiones técnicas. Las 7 suites de pruebas del monorepo mantienen operativas más de 415 pruebas automatizadas pasando de manera ininterrumpida.
4. Nivel de Riesgo Operativo: Clasificado en **Riesgo Operativo Medio**, originado no por deficiencias de codificación, sino por las dependencias externas del entorno de demostración (cuotas de Render Free Tier, datos de potencia calculados sintéticamente y políticas RLS permisivas ya verificadas en Supabase y pendientes de restringir).

10.3. Recomendaciones Mandatorias para la Operación y Sustentación

Para garantizar una sustentación académica exitosa y sentar las bases de la transición hacia la siguiente fase de desarrollo, el Auditor Principal formula cuatro recomendaciones esenciales: 1. Procedimiento de Precalentamiento (Keep-Alive): En toda demostración o sustentación ante el jurado, el equipo debe enviar peticiones de precalentamiento a los servicios de Render con al menos 25 minutos de anticipación, mitigando el retardo de 60 segundos por arranque en frío. 2. Transparencia y Honestidad Científica: Mantener en toda presentación, diapositiva e informe que los datos corresponden a un entorno de data center simulado y que el cálculo energético obedece a un modelo determinista, lo que enaltece el valor metodológico del proyecto. 3. Verificación Mandatoria en Consola Supabase: El responsable de base de datos debe constatar en el editor SQL de Supabase la eliminación definitiva de la columna password en la tabla usuario y la remoción de directivas permisivas cmd: ALL para el rol público. 4. Ejecución del Roadmap de Olas 2 y 3: Continuar con la implementación de las tareas planificadas en los Sprints 7 y 8 (persistencia de aprobación de recomendaciones HU-07, reporte PDF formal HU-13 y rate limiting en Gateway) para alcanzar la meta proyectada del 97.6 % de apego global.

11. Conclusiones y qué aprendimos

El desarrollo de la sesión y la aplicación de los marcos FURPS+ y OWASP Top 10 dejan aprendizajes de alto valor formativo y profesional para el equipo de ingeniería:

1. Proceso de Software Incremental y Preservación de Funcionalidad: Aprendimos que la calidad no se logra mediante reescrituras masivas ni alteraciones disruptivas, sino a través de incrementos delgados, controlados y reversibles (olas de mejora). La adopción de barreras de regresión basadas en suites automatizadas (415+ tests) brindó la confianza requerida para modificar archivos de configuración crítica y código de marcado sin romper los microservicios existentes.
2. Producto de Software Web en Arquitectura Hexagonal: Se consolidó la importancia práctica de desacoplar el núcleo de negocio de los adaptadores de infraestructura. El uso de contratos OpenAPI rigurosos, formatos de error estándar (RFC 7807 problem+json, aplicado en Gateway, Data Processing y Monitoring; Prediction queda pendiente de migrar) y un API Gateway perimetral permitió que microservicios políglotas (Java, Python, Go) convivan de manera armoniosa y admitan implementaciones alternativas (como el gateway ligero en Go) sin alterar la experiencia del cliente web.
3. Criterios de Calidad de Software y Evaluación Crítica: La aplicación del modelo FURPS+ enseñó a valorar las dimensiones no funcionales (Usabilidad, Confiabilidad, Rendimiento y Mantenibilidad) al mismo nivel de exigencia que las funcionalidades de negocio. Comprendimos que la honestidad experimental es un valor superior: reajustar una sobrecalificación inicial del 96 % al 85 % real refuerza la solidez técnica del proyecto ante cualquier auditoría externa.
4. Herramientas de Evaluación y Seguridad Aplicada: El análisis bajo OWASP Top 10:2025 reveló que la seguridad debe concebirse en profundidad, incluido el manejo de condiciones excepcionales (A10:2025) con comportamiento fail-closed. Aprendimos a blindar la cadena de suministro en CI/CD mediante hashes SHA inmutables, a proteger a los usuarios frente a alteraciones de CDN con Subresource Integrity (SRI) y a configurar cabeceras defensivas en Nginx para mitigar vectores de explotación como Clickjacking y MIME-sniffing.
5. Colaboración Sinergica con Agentes de Inteligencia Artificial: La interacción entre Claude Code y Antigravity CLI demostró el potencial de los agentes autónomos en tareas complejas de auditoría, redacción documental y verificación técnica, siempre que exista una supervisión humana rigurosa que guíe los criterios de aceptación, examine los diffs de código y valide la ausencia de alucinaciones contra el repositorio real.

12. Anexos y trazabilidad en el repositorio

Para fines de verificación, auditoría y trazabilidad académica, todos los artefactos documentales, archivos de configuración y suites de código citados en este informe se encuentran versionados en el repositorio oficial de GitHub del proyecto:

- Repositorio oficial: https://github.com/woshtsu/green-ai-workspace-cld
- Versión del repositorio que corresponde a este informe: etiqueta semana13-entrega (rama test): https://github.com/woshtsu/green-ai-workspace-cld/tree/semana13-entrega
- Licencia: MIT (LICENSE)
- Matriz de Calidad en formato Excel: exportados/semana13/MODELO-FURPSPLUS-Green-AI.xlsx

Relación de Entregables del Ciclo de Calidad y Seguridad (01–08)

1. docs/calidad/01_auditoria_furps.md
   Auditoría Integral de Calidad de Software bajo el Modelo FURPS+ (Línea base inicial: 85.00 %; reajustada: 84.0 %).
   Enlace en repositorio: https://github.com/woshtsu/green-ai-workspace-cld/blob/semana13-entrega/docs/calidad/01_auditoria_furps.md
2. docs/calidad/02_plan_furps.md
   Plan Integral de Mejora de Calidad de Software (Estrategia en tres olas y barreras de regresión).
   Enlace en repositorio: https://github.com/woshtsu/green-ai-workspace-cld/blob/semana13-entrega/docs/calidad/02_plan_furps.md
3. docs/calidad/03_auditoria_implementacion_furps.md
   Auditoría Independiente de Implementación FURPS+ (Puntaje verificado post Ola 1: 87.00 %; reajustado: 86.0 %).
   Enlace en repositorio: https://github.com/woshtsu/green-ai-workspace-cld/blob/semana13-entrega/docs/calidad/03_auditoria_implementacion_furps.md
4. docs/calidad/04_auditoria_owasp.md
   Auditoría de Seguridad de Aplicaciones Web y APIs según OWASP Top 10:2021 (Score inicial: 80.5 / 100; reclasificado a 2025: 79.0 / 100).
   Enlace en repositorio: https://github.com/woshtsu/green-ai-workspace-cld/blob/semana13-entrega/docs/calidad/04_auditoria_owasp.md
5. docs/calidad/05_plan_owasp.md
   Plan de Remediación de Vulnerabilidades de Seguridad OWASP Top 10 (Roadmap en tres olas).
   Enlace en repositorio: https://github.com/woshtsu/green-ai-workspace-cld/blob/semana13-entrega/docs/calidad/05_plan_owasp.md
6. docs/calidad/06_auditoria_implementacion_owasp.md
   Auditoría Independiente de Implementación de Seguridad OWASP (Score verificado post Ola 1: 85.0 / 100 bajo 2021; 83.5 / 100 bajo 2025).
   Enlace en repositorio: https://github.com/woshtsu/green-ai-workspace-cld/blob/semana13-entrega/docs/calidad/06_auditoria_implementacion_owasp.md

7. docs/calidad/07_auditoria_final.md
   Dictamen Final Integral de Calidad y Seguridad de Software («Apto para producción con observaciones»).
   Enlace en repositorio: https://github.com/woshtsu/green-ai-workspace-cld/blob/semana13-entrega/docs/calidad/07_auditoria_final.md
8. docs/calidad/08_reclasificacion_owasp_2025.md
   Reclasificación de la auditoría OWASP a Top 10:2025 y evaluación de A10:2025.
   Enlace en repositorio: https://github.com/woshtsu/green-ai-workspace-cld/blob/semana13-entrega/docs/calidad/08_reclasificacion_owasp_2025.md

Documentos Técnicos Complementarios de Evidencia

- docs/evidencias/semana13-ola1-verificacion.md: Registro de comandos y resultados de la verificación técnica de la Ola 1.
- entregables/guia-de-uso/GUIA-RAPIDA-USUARIO.md: Manual operativo en 8 flujos de <= 3 pasos para el usuario final (M-U07).
- docs/GUIA-DESPLIEGUE-RENDER.md: Procedimiento de despliegue contenerizado a costo cero en Render.
- docs/HANDOFF.md: Bitácora de ingeniería, arquitectura hexagonal, inventario de pruebas y directrices de honestidad experimental.

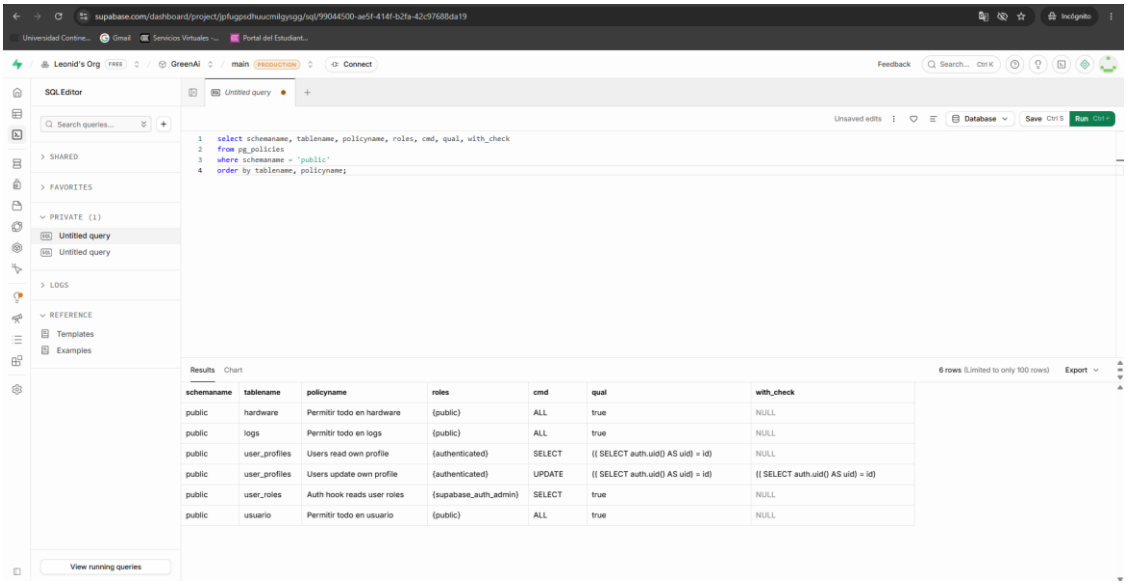
Evidencias pendientes (se entregarán si se solicitan)

Quedan pendientes de captura: S44 (rechazo de acción con rol VIEWER, M-F12); S50a a S50d (resúmenes de pytest de Prediction y Data Processing, go test del Gateway Go y mvn test de Monitoring y Gateway; respaldan M-R04 «415+ pruebas», ejecutadas en el equipo de Ricardo); y la memoria en Render (M-P02: el plan gratuito no expone la métrica; se usa el registro de 122 MiB, S34).

13. Anexo de evidencias de la matriz

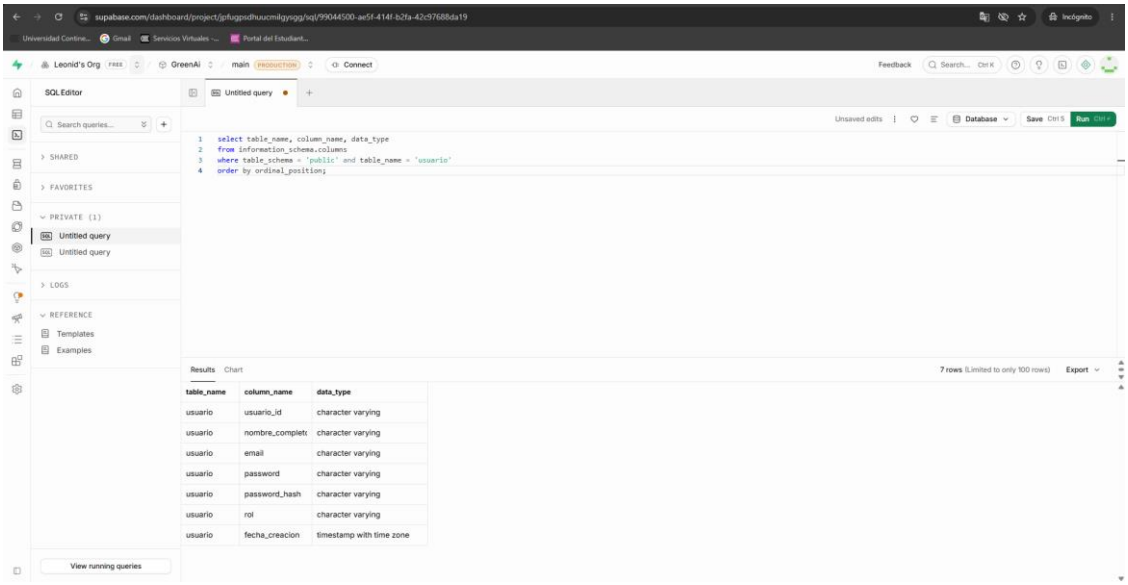
Cada fila de la matriz del §7.1 cita al final su evidencia (Sxx). Las capturas S01, S04, S05, S06, S07, S08, S09, S10, C303 y C304 están insertadas en el §9 y solo se citan; las demás se presentan a continuación. El mapa completo, con las salidas de cada comando, está en docs/evidencias/semana13-mapa-evidencias.md.

13.1. S11



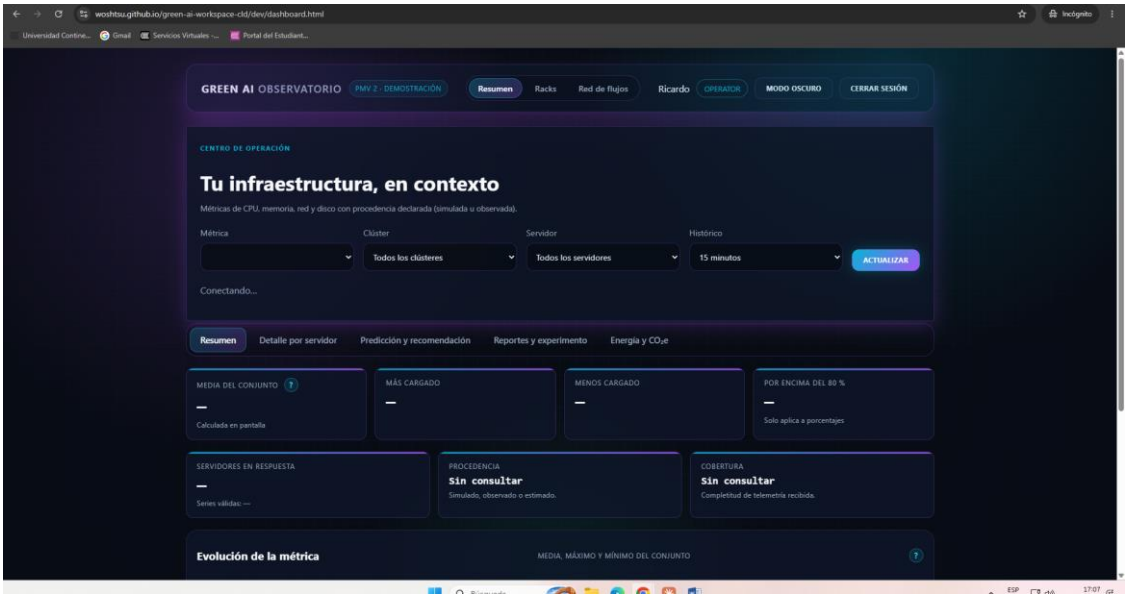
S11 — OW-A01 (OW-01), M-F13, M-R05: consola de Supabase con las políticas «Permitir todo» (ALL, true) para el rol public en hardware, logs y usuario; hallazgo verificado el 12/11/2026

13.2. S12



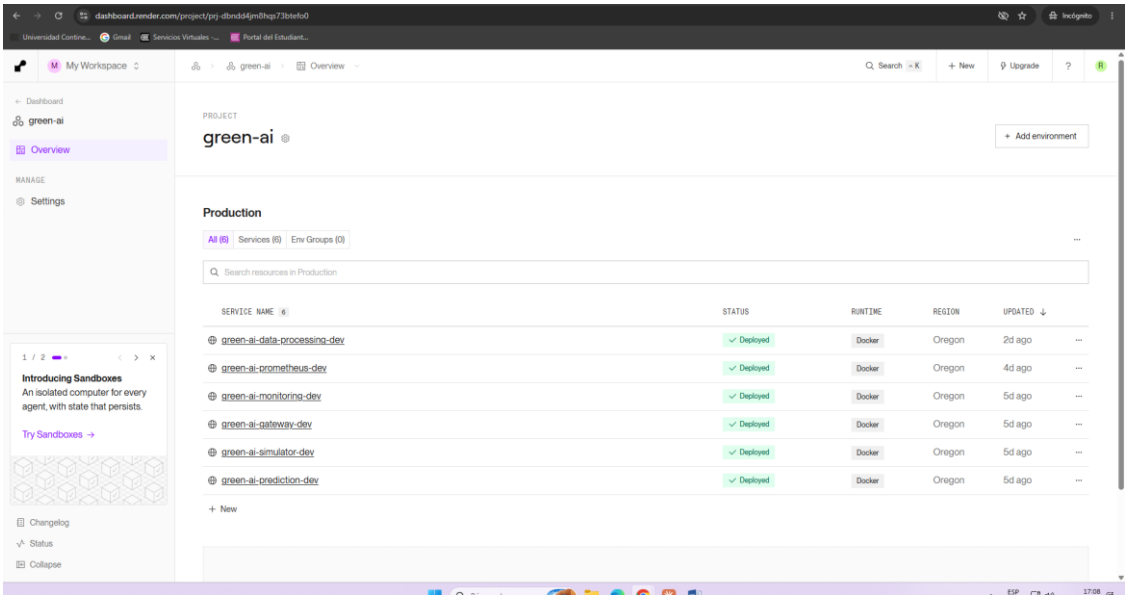
S12 — OW-A04 (OW-03): consola de Supabase con las columnas password y password_hash en la tabla usuario; hallazgo verificado el 12/11/2026

13.3. S13



S13 — M-F13, M-F12 (apoyo), M-U01, M-U02: inicio de sesión real como Ricardo Porras Veli con rol OPERATOR

13.4. S14



S14 — M-PL03: panel de Render con los 6 servicios en plan gratuito (Free)

13.5. S15

M-F07 - Aprobación de recomendaciones no persistida — ejecutado el 12/11/2026 17:17

```
PS C:\green-ai> grep -rn aprobar|rechazar|approv + executionAllowed (software/frontend, software/prediction/src)
--- 1) botones/flujo de aprobar o rechazar (frontend y prediction):
grep exit=1 (1 = sin coincidencias)

--- 2) solo existe la bandera defensiva executionAllowed:
software/frontend/recommendations.js:58:      if (!statuses[result.status] || !actions[result.action] || result.executionAllowed !== false
software/prediction/src/service/recommendations.py:112:    executionAllowed: Literal[False] = False

--- 3) tabla/endpoint de decisiones persistidas:
grep exit=1 (1 = sin coincidencias)
```

S15 — M-F07: búsqueda sin coincidencias de aprobar, rechazar y approv en frontend y prediction; solo existe la bandera executionAllowed: Literal[False] y no hay tabla ni endpoint de decisiones

13.6. S16

M-F08 - Potencia modelada, no medida (energy.js) — ejecutado el 12/11/2026 17:17

```
PS C:\green-ai> grep -n 'MODELO lineal|IDLE_FRACTION|modelPower' software/frontend/energy.js
2:   La potencia a partir de la CPU es un MODELO lineal (P = P_idle + (P_max − P_idle) × CPU), no una medición. */
7:  const IDLE_FRACTION = 0.4;           // fracción de P_max consumida en reposo (supuesto visible en la interfaz)
11:  function modelPower(cpuRatio, maxWatts, idleFraction = IDLE_FRACTION) {
15:    return idle + (maxWatts - idle) * clamped;
---
lecturas de sensores/hardware real en energy.js:
grep exit=1 (1 = sin coincidencias)
```

S16 — M-F08: energy.js modela P = P_idle + (P_max − P_idle) × CPU con IDLE_FRACTION = 0.4; no hay lectura de sensores

13.7. S17

M-F11 - Reporte histórico solo JSON/CSV, sin PDF — ejecutado el 12/11/2026 17:17

```
PS C:\green-ai> grep routes.py (formatos) + grep -rli reportlab|fpdf|weasyprint|jspdf... software/
--- endpoint de reporte: formatos aceptados
93:    "/reports/historical-summary",
100:                "text/csv": {"schema": {"type": "string"}},
112:    format: Literal["json", "csv"] = Query("json"),
123:        return Response(content=summary, media_type="text/csv")

--- generacion de PDF en software/ (reportlab, fpdf, weasyprint, pdfkit, jspdf, iText, application/pdf):
grep exit=1 (1 = sin coincidencias)
```

S17 — M-F11: /reports/historical-summary solo acepta json y csv; no hay librería ni tipo MIME PDF en software/

13.8. S18

M-R05 - SQLite aísla desarrollo; sin prueba de carga remota — ejecutado el 12/11/2026 17:17

```
PS C:\green-ai> sed/grep compose.sqlite-dev.yml, compose.yaml + grep herramientas de carga
--- compose de desarrollo con SQLite (aislado de Supabase):
# No sustituye a Supabase: sin Auth, RLS ni procedimientos. El servicio rechaza este perfil con APP_ENV=production.
software/infraestructura/compose.sqlite-dev.yml:10:      HISTORICAL_SOURCE: sqlite
software/infraestructura/compose.sqlite-dev.yml:11:      SQLITE_PATH: /data/greenai-dev.sqlite
software/infraestructura/compose.sqlite-dev.yml:12:      SUPABASE_ENABLED: "false"
software/data-processing/compose.yaml:7:      SUPABASE_ENABLED: ${SUPABASE_ENABLED:-false}

--- herramientas de prueba de carga (locust, k6, jmeter, vegeta) en software/ y docs/evidencias:
grep exit=1 (1 = sin coincidencias)

--- HANDOFF: RLS permisiva sin corregir
151:- **Supabase con RLS permisiva (según el snapshot `software/simulator/docs/assets/esquema-verificado.json`, 2026-09
```

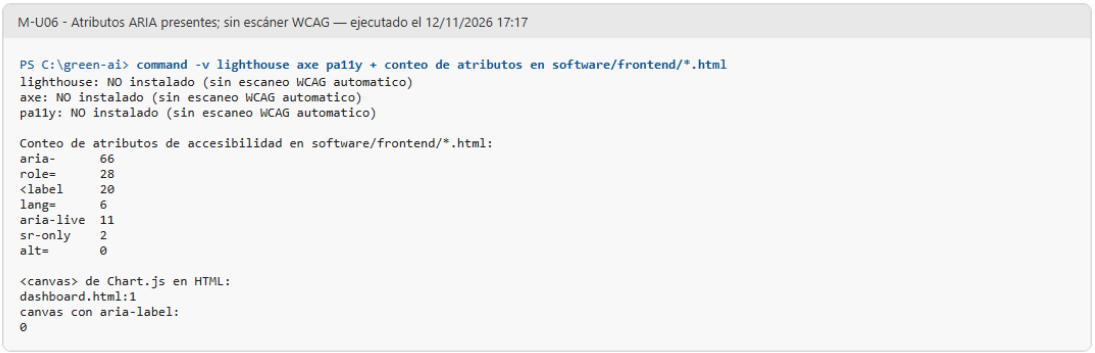
S18 — M-R05: compose.sqlite-dev.yml aísla el desarrollo (HISTORICAL_SOURCE: sqlite, SUPABASE_ENABLED: false); no hay herramientas ni resultados de prueba de carga

13.9. S19



S19 — M-S01: runner.py del Simulator (líneas 5-10) importa adaptadores; en data-processing la capa application no importa infraestructura

13.10. S20



S20 — M-U06: lighthouse, axe y pa11y no están instalados (sin escaneo WCAG); recuento aria- 66, role= 28, label 20, lang= 6, aria-live 11; el único canvas (dashboard.html) no tiene aria-label. El recuento no equivale a conformidad WCAG

13.11. S21

OW-A06 - Sin rate limiting en los gateways — ejecutado el 12/11/2026 17:17

```
PS C:\green-ai> grep -rn ratelimit|limiter|throttle|429 software/gateway-go ; grep RequestRateLimiter software/gateway ; filtros de rutas
--- rate limit / throttle / limiter en gateway-go (codigo, sin pruebas):
grep exit=1 (1 = sin coincidencias)

--- idem en Spring Cloud Gateway (software/gateway):
grep exit=1 (1 = sin coincidencias)

--- filtros definidos en las rutas del gateway Spring:
12:                - Path=/api/monitoring/v1/metrics/catalog,/api/monitoring/v1/metrics/current,/api/monitoring/v1/metri
13:                - Method=GET
14:              filters:
15:                - StripPrefix=3
16:                - PrefixPath=/api/v1
20:                - Path=/api/processing/v1/metrics/history,/api/processing/v1/historical-logs,/api/processing/v1/predi
21:                - Method=GET
22:              filters:
23:                - StripPrefix=3
24:                - PrefixPath=/api/v1
30:                - Path=/api/prediction/v1/predictions
31:                - Method=POST
```

S21 — OW-A06 (OW-05): cero coincidencias de limitadores de tasa en gateway-go y en el Gateway Spring

13.12. S22

OW-A07 - Sesión persistida por supabase-js (localStorage por defecto) — ejecutado el 12/11/2026 17:18

```
PS C:\green-ai> grep createClient/persistSession app.js ; grep storage:|sessionStorage|cookie ; grep localStorage (solo nombre de clave)
--- cliente Supabase en app.js (opciones de sesion):
14:  if (!supabaseUrl || !supabasePublishableKey || !window.supabase?.createClient) {
17:  supabaseClient = window.supabase.createClient(supabaseUrl, supabasePublishableKey, {
18:    auth: { flowType: 'pkce', autoRefreshToken: true, persistSession: true, detectSessionInUrl: true }

--- opcion storage personalizada (sessionStorage/cookies) en el frontend:
grep exit=1 (1 = sin coincidencias -> usa el localStorage por defecto de supabase-js)

--- usos de localStorage en el frontend (solo nombre de clave):
software/frontend/appearance.js:4:localStorage.getItem('greenai-theme'
software/frontend/appearance.js:17:localStorage.setItem('greenai-theme'
```

S22 — OW-A07 (OW-09): createClient con persistSession y sin opción storage (supabase-js usa localStorage); el código propio solo guarda greenai-theme; no se leyó ningún valor de sesión

13.13. S23

OW-A09 - 401/403 sin registro de auditoría en gateway-go — ejecutado el 12/11/2026 17:18

```
PS C:\green-ai> grep StatusUnauthorized|StatusForbidden server.go ; grep slog|audit gateway-go ; sed unauthorized()
--- puntos donde gateway-go responde 401/403:
129:            problem(w, http.StatusForbidden, "Forbidden", "Tu cuenta no tiene permiso para esta operación.")
142:            w.WriteHeader(http.StatusForbidden)
154:            w.WriteHeader(http.StatusForbidden)
160:                    w.WriteHeader(http.StatusForbidden)
234:    problem(w, http.StatusUnauthorized, "Unauthorized", "Inicia sesión con un token válido.")

--- registro (log/slog/audit) en gateway-go, codigo no-test:
software/gateway-go/cmd/gateway/main.go:9:      "log/slog"
software/gateway-go/cmd/gateway/main.go:30:             slog.Error("arranque fallido", "error", err)
software/gateway-go/cmd/gateway/main.go:66:     slog.Info("gateway escuchando", "port", port)

--- cuerpo de la funcion unauthorized():
func unauthorized(w http.ResponseWriter) {
        w.Header().Set("WWW-Authenticate", "Bearer")
        problem(w, http.StatusUnauthorized, "Unauthorized", "Inicia sesión con un token válido.")
}
```

S23 — OW-A09 (OW-12): server.go responde 401 y 403, pero el único slog de gateway-go es de arranque; sin registro de auditoría

13.14. S24

OW-A10 - Pruebas fail-closed existen; sin recover() en gateway-go (go no instalado) — ejecutado el 12/11/2026 17:18

```
PS C:\green-ai> command -v go ; grep ^func Test... server_test.go ; grep recover() ; grep ControllerAdvice software/gateway
go test:
go NO instalado en esta maquina -> pruebas no ejecutadas (solo inspeccion estatica)

--- pruebas A10 existentes (definicion):
407:func TestUpstreamTimeoutIs504(t *testing.T) {
419:func TestUpstreamDownIs503WithoutInternalDetails(t *testing.T) {
433:func TestJWKSUnavailableIs503AndNeverAccepts(t *testing.T) {

--- recover() en gateway-go (middleware de panics):
grep exit=1 (1 = sin coincidencias)

--- manejador global / error.include en Spring Cloud Gateway:
grep exit=1 (1 = sin coincidencias)
```

S24 — OW-A10 (OW-13): Go no está instalado, por lo que las 3 pruebas fail-closed de server_test.go no se ejecutaron; sin recover() en gateway-go ni manejador global en el Gateway Spring

13.15. S25

M-P01 - Tiempos de respuesta en Render (curl, 2 intentos por endpoint) — ejecutado el 12/11/2026 17:18

```
PS C:\green-ai> curl -s -o /dev/null --max-time 120 -w ... <endpoint de salud> (x2)
17:17:05 intento 1 https://green-ai-gateway-dev.onrender.com/actuator/health
  HTTP 200  total=0.473251s  conexion=0.233198s  primer_byte=0.473136s
17:17:06 intento 2 https://green-ai-gateway-dev.onrender.com/actuator/health
  HTTP 200  total=0.250499s  conexion=0.015252s  primer_byte=0.250361s
17:17:06 intento 1 https://green-ai-prediction-dev-zxly.onrender.com/health/ready
  HTTP 200  total=9.302610s  conexion=8.484356s  primer_byte=9.302493s
17:17:16 intento 2 https://green-ai-prediction-dev-zxly.onrender.com/health/ready
  HTTP 200  total=0.774555s  conexion=0.274866s  primer_byte=0.774485s
```

S25 — M-P01: Gateway 0,47 s y 0,25 s; Prediction primer intento 9,3 s (conexión 8,48 s) y segundo 0,77 s. El primer intento no es un arranque en frío completo

13.16. S26

M-S02 / M-S04 - Contratos OpenAPI versionados y monorepo sin gitlinks — ejecutado el 12/11/2026 17:18

```
PS C:\green-ai> git ls-files | grep -i -E 'openapi|swagger'
docs/historico/semana10/legacy/green-ai-recommendations/openapi.json
docs/historico/trabajos-previos/evidencias-del-sistema/capturas-swagger/Prediction-1.jpg
docs/historico/trabajos-previos/evidencias-del-sistema/capturas-swagger/Prediction-2.jpg
entregables/informe-corregido/evidencias-nube/captura-C305-swagger-prediction.png
software/data-processing/contracts/data-processing-v1.openapi.json
software/data-processing/scripts/export_openapi.py
software/gateway/src/main/resources/static/openapi/gateway-v0.1.yaml
software/gateway/src/main/resources/static/openapi/recommendations-v1.json
software/infraestructura/compose.swagger.yaml
software/monitoring/src/main/resources/static/openapi/monitoring-v0.1.json
software/monitoring/src/test/java/pe/edu/continental/greenai/monitoring/contract/OpenApiAssertions.java
software/prediction/contracts/prediction.openapi.json
software/prediction/contracts/recommendations-gateway.openapi.json
software/prediction/scripts/export_openapi.py

14
PS C:\green-ai> git rev-parse --show-toplevel ; git ls-files -s | awk '$1=="160000"' | wc -l ; git ls-files .gitmodules | wc -l
$ git rev-parse --show-toplevel
D:/Green-ai/green-ai-workspace-cld

$ git ls-files -s | awk $1==160000 | wc -l   (gitlinks/submodulos)
0

$ git ls-files .gitmodules
0
(0 = sin .gitmodules)
```

S26 — M-S02, M-S04: 14 rutas versionadas con openapi/swagger (la lista incluye archivos históricos e imágenes) y monorepo con 0 gitlinks y 0 .gitmodules

13.17. S27

M-PL01 / M-PL02 / M-PL06 - Guías de despliegue/arranque y docker-compose — ejecutado el 12/11/2026 17:18

```
PS C:\green-ai> ls docs/GUIA-* entregables/guia-de-uso ; sed -n 22,38p README.md
docs/GUIA-DESPLIEGUE-RENDER.md
docs/GUIA-GITHUB-PAGES-Y-CI.md

entregables/guia-de-uso/:
GUIA-RAPIDA-USUARIO.md

## Pruebas por servicio (Windows/PowerShell, desde la raíz)
```powershell
cd software\monitoring; .\mvnw.cmd -B verify; cd ..\..
cd software\gateway; .\mvnw.cmd -B verify; cd ..\..
cd software\simulator; uv run --frozen python -m unittest discover -s tests; cd ..\..
cd software\data-processing; uv venv .venv; uv pip install --python .venv\Scripts\python.exe -r requirements.t
cd software\prediction; uv venv .venv --python 3.11.9; uv pip install --python .venv\Scripts\python.exe -r req
cd software\frontend; node --test tests/frontend.test.cjs tests/prediction.test.cjs tests/recommendations.test
```

Ejecución local con Docker
```powershell
Copy-Item software\frontend\config.example.js software\frontend\config.local.js   # editar; gatewayBaseUrl htt
docker compose -f software/infraestructura/docker-compose.yml -f software/infraestructura/compose.swagger.yaml
```
Frontend http://127.0.0.1:13001 · Gateway :18081 · Prometheus :19090 · Swagger Prediction :18002/docs.
Puertos distintos del workspace antiguo para que ambos puedan coexistir.

docs/CONTINUAR-EN-NUEVO-CHAT.md

PS C:\green-ai> servicios en docker-compose.yml y overlays
servicios definidos en software/infraestructura/docker-compose.yml:
  simulator:
  prometheus:
  monitoring:
  data-processing:
  prediction:
  gateway:
  frontend:

overlays:
software/infraestructura/compose.contract-postgres.yml
software/infraestructura/compose.gateway-go.yml
software/infraestructura/compose.observability.yml
software/infraestructura/compose.sqlite-dev.yml
software/infraestructura/compose.swagger.yaml
```

S27 — M-PL01, M-PL02, M-PL06: guías de despliegue y de arranque existentes, y docker-compose.yml con 7 servicios y 5 overlays; no se ejecutó docker compose up

13.18. S28

M-R01 / M-R02 - Formato de error por servicio y rutas de salud — ejecutado el 12/11/2026 17:18

```
PS C:\green-ai> grep problem+json|ProblemDetail por servicio ; grep INFERENCE_ERROR
Formato de error por servicio (codigo de produccion, sin pruebas):
data-processing  problem+json/ProblemDetail: 2
gateway-go       problem+json/ProblemDetail: 1
monitoring       problem+json/ProblemDetail: 5
prediction       problem+json: 0 | sobre propio {code,message} (_error_payload): 3

Respuesta 500 de Monitoring:
46:        return ResponseEntity.status(status).contentType(MediaType.APPLICATION_PROBLEM_JSON).body(body);
Respuesta 500 de Prediction (app.py):
71:    return 500, "INFERENCE_ERROR", "Inference failed", {}
89:        content=_error_payload("INFERENCE_ERROR", "Inference failed"),

PS C:\green-ai> grep rutas /health, /actuator/health, -healthcheck
rutas de salud en el codigo:
software/prediction/src/service/app.py:94:@app.get("/health")
software/prediction/src/service/app.py:112:@app.get("/health/live")
software/prediction/src/service/app.py:117:@app.get("/health/ready")
software/data-processing/infrastructure/adapters/input/api/routes.py:33:@router.get("/health", tags=["Health"])
software/gateway-go/internal/httpapi/server.go:177:     case p == "/actuator/health":
software/gateway-go/internal/httpapi/server.go:179:     case p == "/actuator/health/liveness" || p == "/actuator/health/readiness":
software/gateway-go/cmd/gateway/main.go:23:     healthcheck := flag.Bool("healthcheck", false, "consulta /actuator/health/liveness y termina
(para Docker)
software/gateway-go/cmd/gateway/main.go:79:     resp, err := c.Get("http://127.0.0.1:" + port + "/actuator/health/liveness")

software/monitoring/src/main/resources/application.yaml:16:        include: health,prometheus
software/monitoring/src/main/resources/application.yaml:19:      probes:
software/gateway/src/main/resources/application.yaml:63:        include: health,info,prometheus
software/gateway/src/main/resources/application.yaml:66:      probes:
```

S28 — M-R01, M-R02: problem+json en data-processing, gateway-go y monitoring; Prediction usa su propio sobre {code, message}; rutas de salud por servicio, sin prueba en ejecución local

13.19. S29

OW-02: puertos publicados en docker-compose.yml — ejecutado el 12/11/2026 17:39

```
PS C:\green-ai> grep -n -A3 "ports:" software/infraestructura/docker-compose.yml
$ grep -n -A3 "ports:" software/infraestructura/docker-compose.yml
13:    ports:
14-      - "127.0.0.1:${MONO_SIMULATOR_PORT:-18000}:8000"
15-    command: >
16-      run
--
30:    ports:
31-      - "127.0.0.1:${MONO_PROMETHEUS_PORT:-19090}:9090"
32-    volumes:
33-      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
--
42:    ports:
43-      - "127.0.0.1:${MONO_MONITORING_PORT:-18080}:8080"
44-    environment:
45-      - MONITORING_PROMETHEUS_URL=http://prometheus:9090
--
82:    ports:
83-      - "127.0.0.1:${MONO_GATEWAY_PORT:-18081}:8081"
84-    environment:
85-      - GATEWAY_MONITORING_BASE_URL=http://monitoring:8080
--
100:    ports:
101-      - "127.0.0.1:${MONO_FRONTEND_PORT:-13001}:3001"
102-    volumes:
103-      # Crear desde software/frontend/config.example.js (gatewayBaseUrl http://127.0.0.1:18081).
```

S29 — OW-A01 (OW-02): 5 servicios de compose con puerto publicado, todos enlazados a 127.0.0.1 (acceso local, no desde la red)

13.20. S30

OW-04: columnas de contrasena en BD_GreenAi.xlsx (valores no impresos) — ejecutado el 12/11/2026 17:38

```
PS C:\green-ai> python openpyxl BD_GreenAi.xlsx (solo cabecera y conteo)
$ python (openpyxl) BD_GreenAi.xlsx: hojas, cabecera, conteo de celdas no vacias en columnas password
archivo: software/data-processing/datasets/raw/synthetic_greenai/BD_GreenAi.xlsx
hoja: Usuarios | cabecera: ['usuario_id', 'nombre_completo', 'email', 'password', 'password_hash', 'rol', 'fecha_creacion']
  filas de datos: 10
  columna 'password' -> celdas no vacias: 10 (valores NO impresos)
  columna 'password_hash' -> celdas no vacias: 10 (valores NO impresos)
hoja: Hardware | cabecera: ['hardware_id', 'usuario_id', 'hostname', 'ip_address', 'cpu_cores', 'ram_gb', 'max_watts', 'ubicacion_rack',
'estado', 'fecha_registro']
  filas de datos: 50
  sin columna tipo password
hoja: Logs | cabecera: ['log_id', 'hardware_id', 'timestamp', 'cpu_utilization_pct', 'ram_utilization_pct', 'temperatura_celsius',
'energia_watts', 'prediccion_watts', 'accion_recomendada', 'cumple_sla']
  filas de datos: 100000
  sin columna tipo password
```

S30 — OW-A04 (OW-04): BD_GreenAi.xlsx, hoja Usuarios, columnas password y password_hash con 10 de 10 celdas no vacías (valores no impresos)

13.21. S31

OW-11: imagenes base en Dockerfiles — ejecutado el 12/11/2026 17:38

```
PS C:\green-ai> grep -n "^FROM" Dockerfiles; conteo de @sha256
$ git ls-files | grep Dockerfile | xargs grep -n "^FROM"
software/data-processing/Dockerfile:1:FROM python:3.14-slim
software/frontend/Dockerfile:1:FROM nginx:1.28.0-alpine
software/gateway-go/Dockerfile:3:FROM golang:1.25.14-alpine@sha256:1ae0735f00daffa3aaf1363a5184c0d2dc55c78e3db4ec70241cdac97bf84b59 AS build
software/gateway-go/Dockerfile:10:FROM gcr.io/distroless/static-
debian12:nonroot@sha256:afa5c872c891853ca7fcf1f12c3edb23f7eeef36189728842dd51042ff57f7ab
software/gateway/Dockerfile:1:FROM maven:3.9.11-eclipse-temurin-21 AS build
software/gateway/Dockerfile:7:FROM eclipse-temurin:21-jre-jammy
software/infraestructura/prometheus/Dockerfile.render:3:FROM prom/prometheus:v3.13.3
software/monitoring/Dockerfile:1:FROM eclipse-temurin:21-jdk-jammy AS build
software/monitoring/Dockerfile:8:FROM eclipse-temurin:21-jre-jammy
software/prediction/Dockerfile:2:FROM python:3.11.9-slim-bookworm AS builder
software/prediction/Dockerfile:20:FROM python:3.11.9-slim-bookworm
software/simulator/Dockerfile:1:FROM python:3.12-slim

coincidencias con @sha256: 1
```

S31 — OW-A08 (OW-11): 11 líneas FROM en los Dockerfiles; solo gateway-go (2) usa @sha256 y el resto solo etiqueta

13.22. S32

M-R03: timeouts configurados — ejecutado el 12/11/2026 17:38

```
PS C:\green-ai> grep Timeout gateway-go (policy/routes.go, httpapi/server.go)
30:     Timeout     time.Duration
34:     defaultTimeout = 6 * time.Second
43:             routes = append(routes, Route{"GET", "/api/monitoring/v1/metrics/" + p, Monitoring, 3, "/api/v1", 0, defaultTimeout})
42:     ConnectTimout time.Duration
55:     if cfg.ConnectTimout == 0 {
56:             cfg.ConnectTimout = time.Second
59:             DialContext:         (&net.Dialer{Timeout: cfg.ConnectTimout}).DialContext,
212:    ctx, cancel := context.WithTimeout(context.WithValue(r.Context(), ctxKey{}, route), route.Timeout)
PS C:\green-ai> grep -i timeout gateway/application.yaml
26:                response-timeout: 15000
38:                response-timeout: 30000
50:                response-timeout: 30000
52:            connect-timeout: 1000
53:            response-timeout: 6s
PS C:\green-ai> grep httpx timeout data-processing
16:        self.timeout_seconds = validate_timeout(timeout_seconds)
31:            with httpx.Client(timeout=self.timeout_seconds) as client:
51:        except httpx.TimeoutException as exc:
```

S32 — M-R03: timeouts configurados en gateway-go (6, 15 y 30 s), Gateway Spring y data-processing; evidencia de configuración en código, no de ejecución

13.23. S33

M-R06: manejo de exito/fallo de la fuente de datos — ejecutado el 12/11/2026 17:38

```
PS C:\green-ai> cd adapters/input; grep -n "raise Source|raise ServiceError|mode=ro" *.py
monitoring_http_adapter.py:52:            raise SourceTimeout("Monitoring no respondió a tiempo.") from exc
monitoring_http_adapter.py:60:                raise SourceTimeout("La fuente de Monitoring excedió el tiempo permitido (504).") from exc
monitoring_http_adapter.py:61:            raise ServiceError(f"Monitoring no está disponible ({status}).") from exc
monitoring_http_adapter.py:63:            raise ServiceError("No fue posible comunicarse con Monitoring.") from exc
sqlite_historical_logs_adapter.py:30:            raise ServiceError("La base SQLite de desarrollo no existe.")
sqlite_historical_logs_adapter.py:31:        # mode=ro: el puerto es de solo lectura, igual que el token de Supabase.
sqlite_historical_logs_adapter.py:32:        return sqlite3.connect(f"{self.path.resolve().as_uri()}?mode=ro", uri=True, timeout=5)
sqlite_historical_logs_adapter.py:53:            raise ServiceError("La base SQLite de desarrollo no está disponible.") from exc
sqlite_historical_logs_adapter.py:55:            raise ServiceError("La base SQLite de desarrollo no está disponible.") from exc
supabase_historical_logs_adapter.py:54:            raise SourceTimeout("Supabase no respondió a tiempo.") from exc
supabase_historical_logs_adapter.py:57:                raise SourceForbidden("La identidad de lectura no tiene acceso a los históricos.") f
supabase_historical_logs_adapter.py:59:                raise SourceTimeout("Supabase no respondió a tiempo.") from exc
supabase_historical_logs_adapter.py:62:            raise ServiceError("Supabase no está disponible.") from exc
supabase_historical_logs_adapter.py:64:            raise ServiceError("No fue posible comunicarse con Supabase.") from exc
PS C:\green-ai> grep -A6 exception_handler(ServiceError) main.py
40:@app.exception_handler(ServiceError)
41-async def source_error(request, exc):
42-    return problem(request, exc.status, exc.code, str(exc))
43-
44-
45-@app.exception_handler(RequestValidationError)
46-async def validation_error(request, exc):
```

S33 — M-R06: los adaptadores mapean los fallos a SourceTimeout, SourceForbidden y ServiceError, y SQLite abre en modo de solo lectura; evidencia de código

13.24. S34

M-P03: imagen de inferencia ligera (registro real) — ejecutado el 12/11/2026 17:38

```
PS C:\green-ai> grep -n GB|MB|MiB docs/evidencias/prediction-imagen-ligera-20261107.md
3:Cambio: `software/prediction/Dockerfile` multi-etapa (entrena con `requirements.lock.txt`; la imagen final instala solo
4:`requirements.serve.txt`), importación diferida de `xgboost` en `src/models/xgboost_model.py`, `--no-compile`,
9:| Tamaño de imagen | 2,85 GB | **510 MB** | `docker images`, Docker Desktop/WSL2 |
10:| Memoria en reposo | 174 MiB | **133 MiB** (contenedor aislado) | `docker stats --no-stream`, una sola medición, sin carga |
11:| Memoria en reposo dentro del stack | 174 MiB | **142 MiB**; con `PYTHONOPTIMIZE=2`: **122 MiB** | `docker stats --no-stream` con el stack
levantado |
13:Pruebas: suite completa de Prediction con el cambio, 100 pruebas sin fallos (entorno completo, Windows). Imagen: `/health/ready` 200
14:y servicio sin `xgboost`, `matplotlib`, `mlflow`, `seaborn` ni `statsmodels`. Stack completo reconstruido con la imagen nueva:
21:Quita docstrings y `assert` en tiempo de ejecución (`src/` no usa `assert`) y desinstala pip/setuptools/wheel de la imagen final. RAM en
reposo en el stack: 142 → **122 MiB** (una medición, sin
```

S34 — M-P03, M-P02: registro del 07/11/2026 (imagen de Prediction de 2,85 GB a 510 MB; RAM de 174 a 133, 142 y 122 MiB); no se volvió a medir en esta sesión

13.25. S35

M-PL04: tabla de rutas del gateway y configuracion del frontend — ejecutado el 12/11/2026 17:38

```
PS C:\green-ai> grep -n "Route{" policy/routes.go
43:             routes = append(routes, Route{"GET", "/api/monitoring/v1/metrics/" + p, Monitoring, 3, "/api/v1", 0, defaultTimeout})
46:             routes = append(routes, Route{"GET", "/api/processing/v1/" + p, DataProcessing, 3, "/api/v1", 0, 15 * time.Second})
48:     routes = append(routes, Route{"POST", "/api/prediction/v1/predictions", Prediction, 2, "", maxBody, 30 * time.Second})
50:             routes = append(routes, Route{"POST", "/api/recommendations/v1/" + p, Prediction, 2, "", maxBody, 30 * time.Second})
65:     return Route{}, false
PS C:\green-ai> grep gatewayBaseUrl / GATEWAY_BASE_URL / supabaseClient (frontend)
6:  gatewayBaseUrl: 'http://127.0.0.1:8081'
5:const GATEWAY_BASE_URL = String(GREEN_AI_CONFIG.gatewayBaseUrl || 'http://127.0.0.1:8081').replace(/\/$/, '');
6:const API_URL = `${GATEWAY_BASE_URL}/api`;
56:  const gateway = new URL(GATEWAY_BASE_URL, location.href);
8:let supabaseClient = null;
12:  if (supabaseClient) return supabaseClient;
14:  if (!supabaseUrl || !supabasePublishableKey || !window.supabase?.createClient) {
17:  supabaseClient = window.supabase.createClient(supabaseUrl, supabasePublishableKey, {
```

S35 — M-PL04: 4 grupos de rutas del gateway y uso de GATEWAY_BASE_URL en el frontend; el inicio de sesión va directo a Supabase Auth

13.26. S36

M-PL05: licencias en el repositorio — ejecutado el 12/11/2026 17:38

```
PS C:\green-ai> git ls-files | grep -i licen; grep -i licen README.md; pom.xml
git ls-files | grep -i licen -> 0 archivos
ls raiz: 0 archivos LICENSE/COPYING
grep -i licen README.md -> 0 coincidencias
pom.xml monitoring:
        <licenses>
                <license/>
        </licenses>
```

S36 — M-PL05: recuento del 12/11/2026 (17:38) que no encontró archivo de licencia versionado; el archivo LICENSE (MIT) se añadió después en la raíz del repositorio

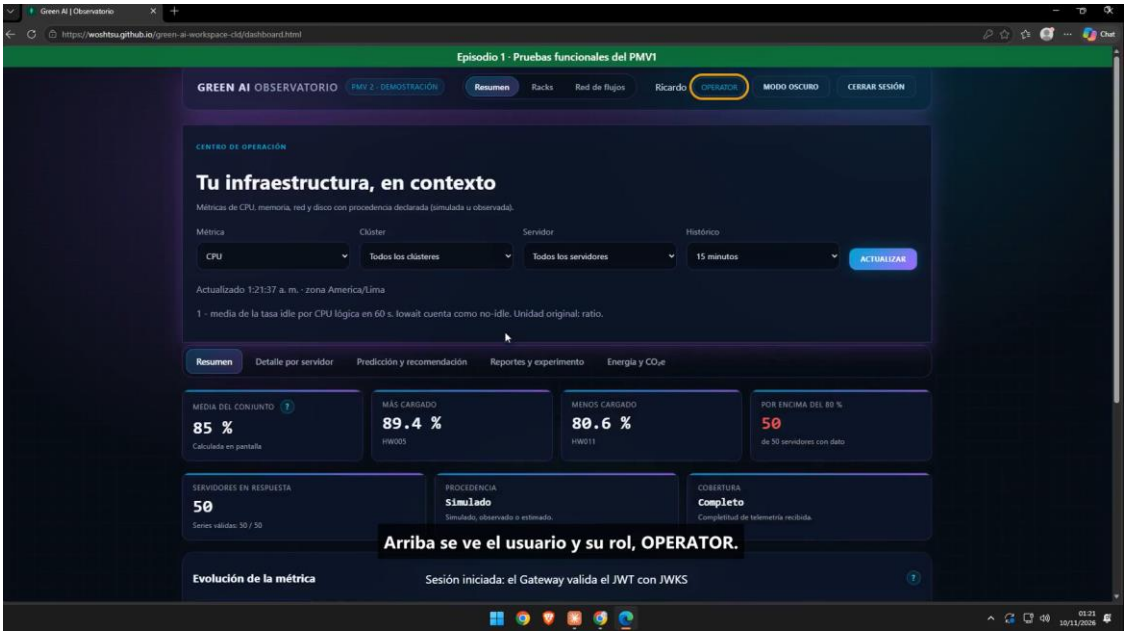
13.27. S37

OW-A05: consultas parametrizadas y validacion — ejecutado el 12/11/2026 17:38

```
PS C:\green-ai> grep extra='forbid'
software/prediction/src/service/recommendations.py:17:    model_config = ConfigDict(extra='forbid', allow_inf_nan=False)
software/data-processing/domain/monitoring_contract.py:17:    model_config = ConfigDict(extra="forbid", allow_inf_nan=False)
PS C:\green-ai> sqlite: parametros ? / PostgREST quoted()
              "WHERE hardware_id = ? AND timestamp >= ? AND timestamp <= ?")
          sql += " AND (timestamp > ? OR (timestamp = ? AND log_id > ?))"
          params += [instant, instant, after[1]]
      sql += " ORDER BY timestamp ASC, log_id ASC LIMIT ?"
def quoted(value):
    # PostgREST quoted filter literals, then httpx handles URL encoding.
    return '"' + value.replace('\\', '\\\\').replace('"', '\\"') + '"'
12:def quoted(value):
38:            params.append(("or", f"(timestamp.gt.{instant},and(timestamp.eq.{instant},log_id.gt.{quoted(after[1])}))"))
PS C:\green-ai> PrometheusQueryBuilder: quote() y lista de metricas permitidas
21:                ? properties.clusterLabel() + "=~" + quote(query.cluster().replace(".", "\\.") + "|")
22:                : properties.clusterLabel() + "=" + quote(query.cluster()));
24:        filters.add(properties.instanceLabel() + (query.resourceId() == null ? "!=\"\"" : "=" + quote(query.resourceId())));
28:            case "node.cpu.utilization" -> {
32:            case "node.memory.used" -> "node_memory_MemTotal_bytes" + selector + " - node_memory_MemAvailable_bytes" + selector;
33:            case "node.network.receive" -> "rate(node_network_receive_bytes_total" + selector + "[1m])";
34:            case "node.network.transmit" -> "rate(node_network_transmit_bytes_total" + selector + "[1m])";
35:            case "node.filesystem.used" -> {
36:                filters.add("fstype=~" + quote(FILESYSTEM_TYPES));
37:                filters.add("mountpoint!~" + quote(EXCLUDED_MOUNTS));
42:            default -> throw MonitoringFailure.invalid("Métrica no permitida.");
47:    private String quote(String value) {
```

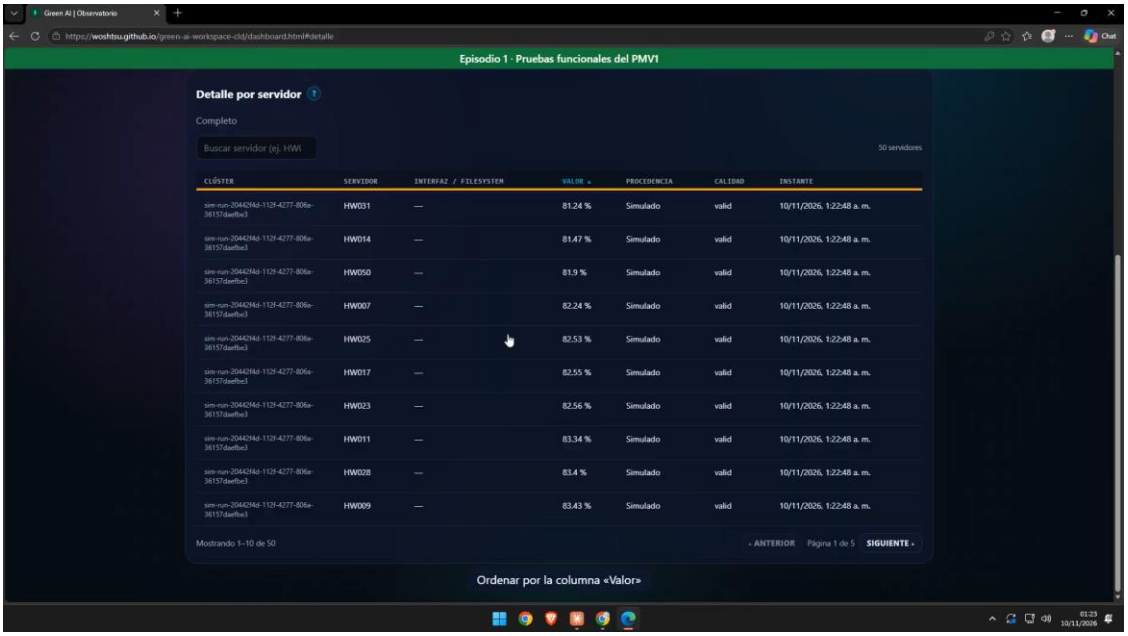
S37 — OW-A05: consultas con parámetros en SQLite, escape en PostgREST, Pydantic con extra forbid y lista blanca de métricas PromQL; evidencia de código, no prueba de ausencia de inyección

13.28. S40



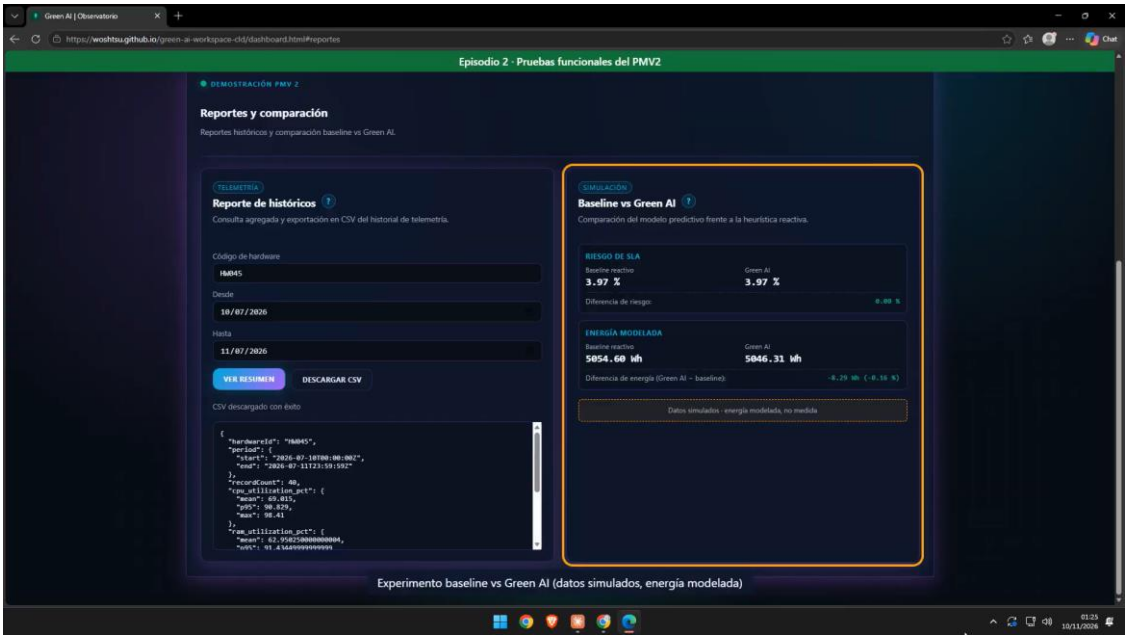
S40 — M-F01, M-U01: visualización de métricas en tiempo real; fotograma del video de sustentación del 10/11/2026

13.29. S41



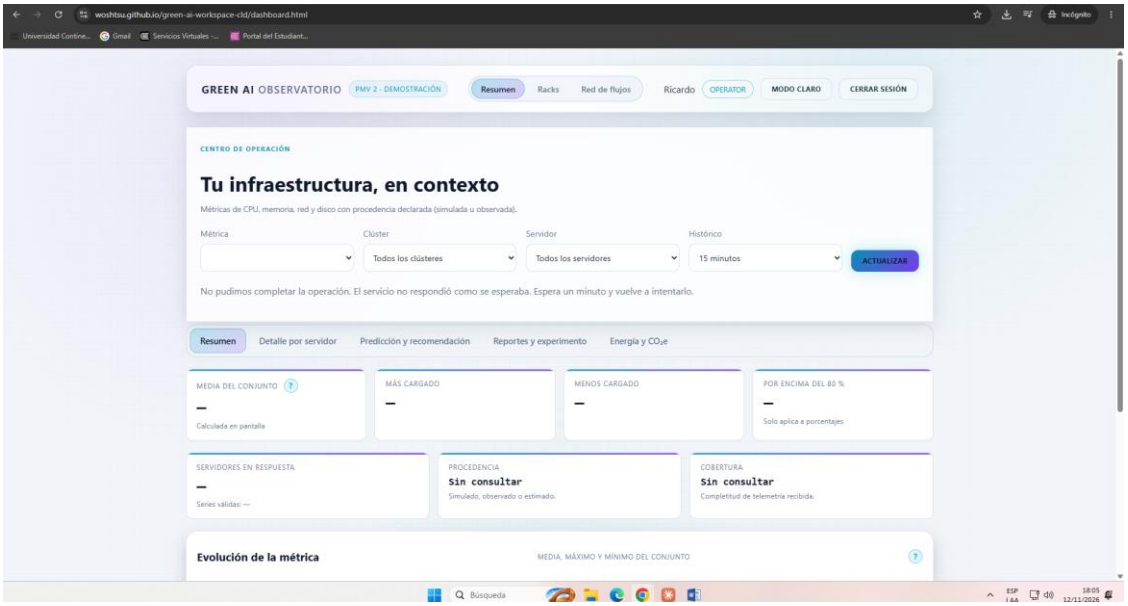
S41 — M-F02: consulta del histórico; fotograma del video de sustentación del 10/11/2026

13.30. S43



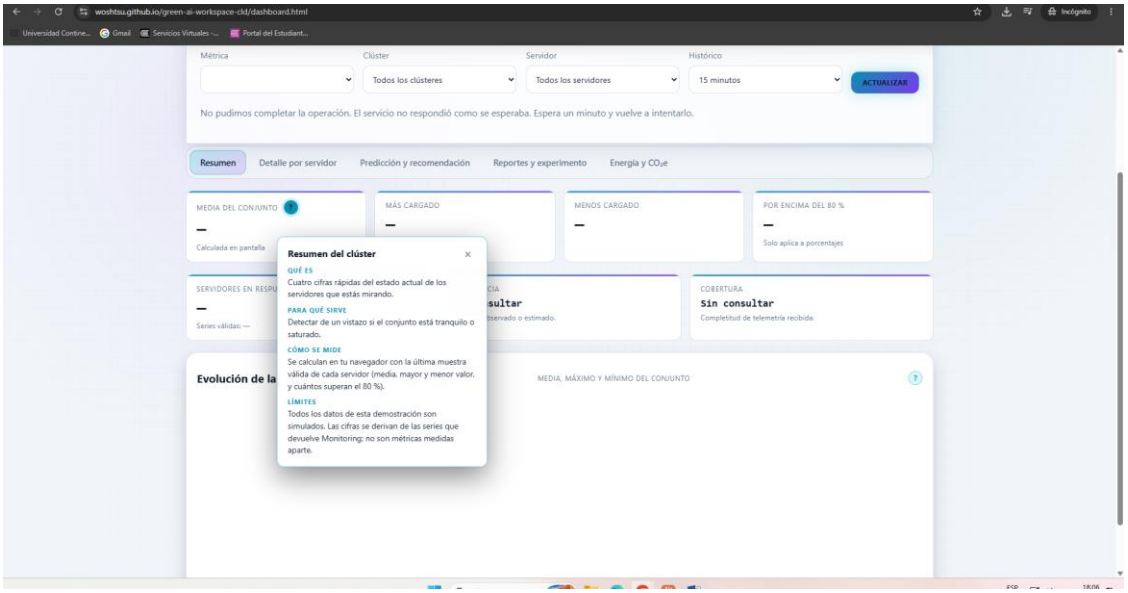
S43 — M-F04, M-F11: experimento de comparación en la interfaz; fotograma del video de sustentación del 10/11/2026

13.31. S45



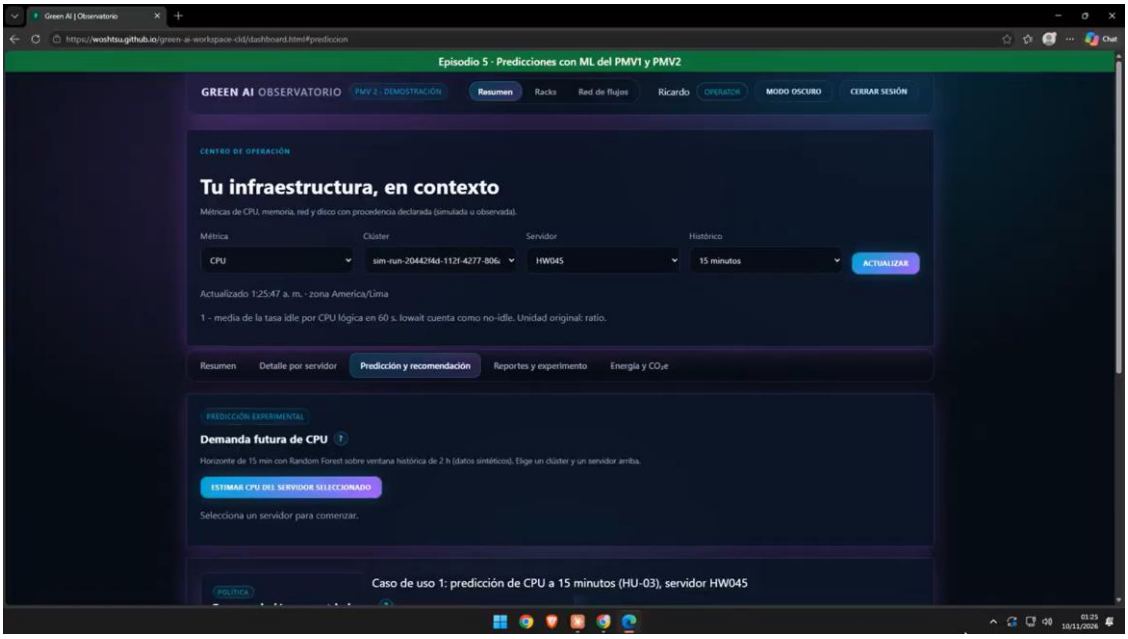
S45 — M-U02, M-U05: modo claro y mensaje de error amigable mientras los servicios de Render despertaban; captura del 12/11/2026

13.32. S46



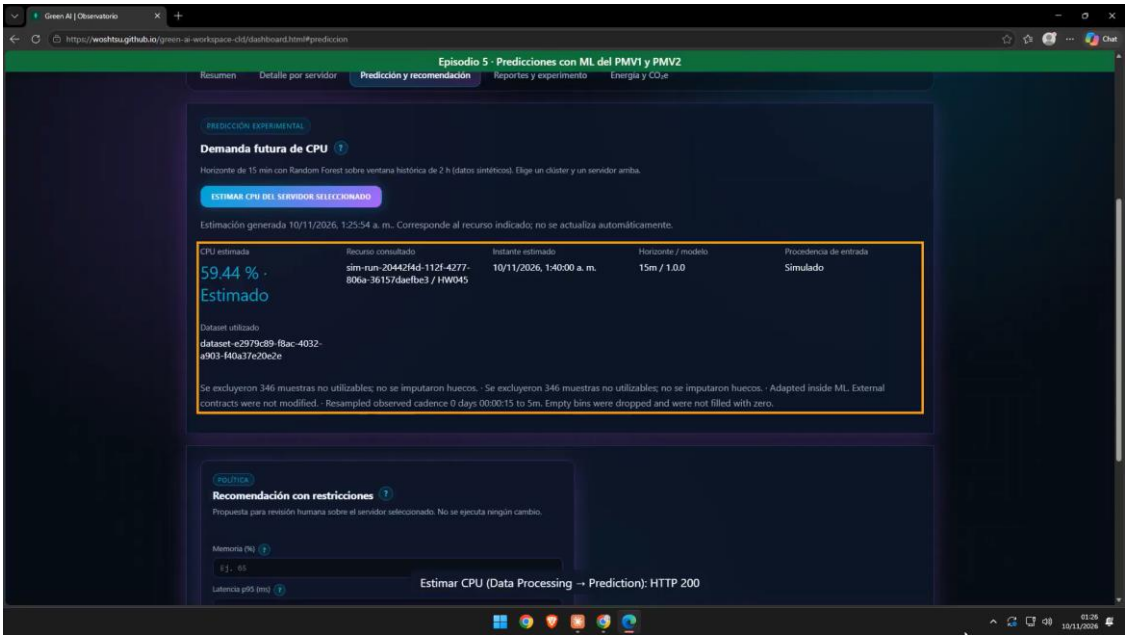
S46 — M-U04: ayuda contextual del botón «?» de «Media del conjunto»; captura del 12/11/2026

13.33. S48a



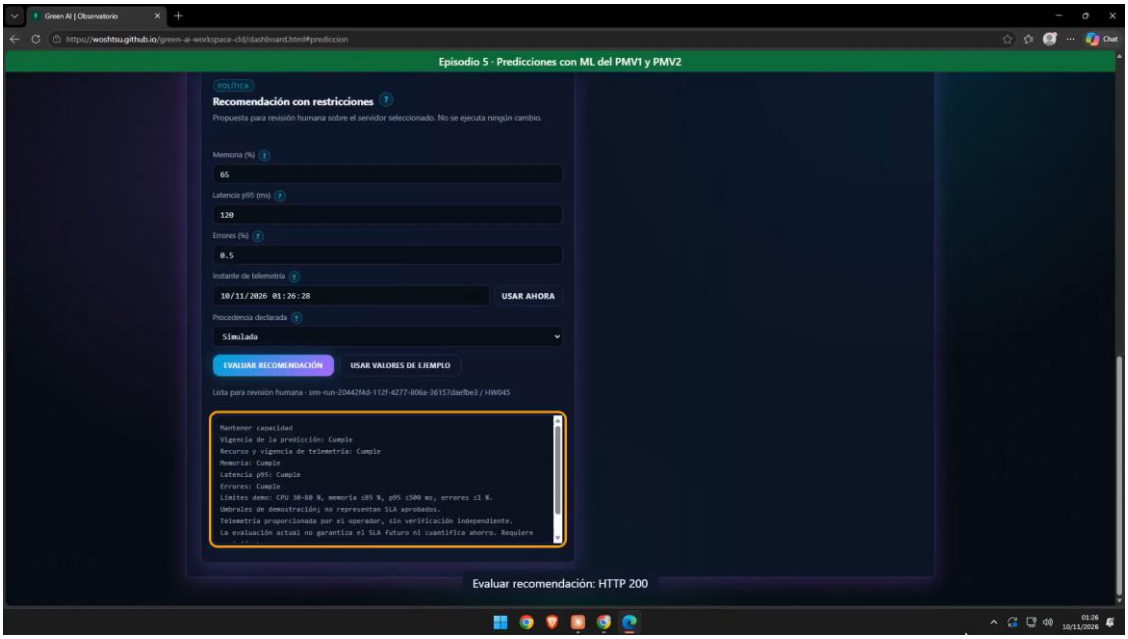
S48a — M-U08: paso 1 (servidor); fotograma del video de sustentación del 10/11/2026

13.34. S48b



S48b — M-U08, M-F03: paso 2 (predicción); fotograma del video de sustentación del 10/11/2026

13.35. S48c



S48c — M-U08, M-F06: paso 3 (recomendación); fotograma del video de sustentación del 10/11/2026